Тема 6

Снижение размерности

Снижение размерности

Постановка задачи
Выделение главных компонентов
Выделение независимых компонентов

Все ли собранные данные полезны?

Вопрос: почему какие-то параметры прецедентов могут быть бесполезными на практике?

Все ли собранные данные полезны?

Вопрос: почему какие-то параметры прецедентов могут быть бесполезными на практике?

Ответ:

- их значения недостоверны (надо было лучше собирать данные);
- их значения не относятся к поставленной задаче (как ни собирай, это не поможет);
- их значения полностью определяются другими параметрами, и в используемых методах они взаимозаменяемы.

Хотелось бы:

- либо отбрасывать ненужные параметры, оставляя только информативные;
- либо поменять набор параметров, чтобы в оставшихся сохранить максимум полезной информации.

Выделение главных компонентов

Principal Component Analysis, PCA

идея: получим из старых признаков новые так, чтобы они стали 1) линейно независимыми и 2) упорядоченными по доле содержащейся информации (т.е. по доле дисперсии данных для собранной выборки).

Пусть выборка состоит из N прецедентов X_i, каждый из которых имеет M числовых признаков: $X_i = (x_{i1}, x_{i2}, ..., x_{iM}), i = 1..N$. Признаки образуют матрицу выборки:

$$Z = \begin{pmatrix} x_{11} & x_{12} & \dots & x_{1M} \\ x_{21} & x_{22} & \dots & x_{2M} \\ \dots & \dots & \dots & \dots \\ x_{N1} & x_{N2} & \dots & x_{NM} \end{pmatrix}$$

В общем случае $M \neq N$; чаще всего $M < N$, но не всегда. Строки иногда могут повторяться.

Зависимости между параметрами

Вопрос: как влияют на матрицу Z линейные зависимости между параметрами (столбцами)?

Зависимости между параметрами

Вопрос: как влияют на матрицу Z линейные зависимости между параметрами (столбцами)?
Ответ: они понижают её ранг.

Попытаемся заменить $(x_{i1}, x_{i2}, ..., x_{iM})$ на другой набор признаков меньшей длины $(\xi_{i1}, \xi_{i2}, ..., \xi_{iK}), K < M$:

$$Q = \begin{pmatrix} \xi_{11} & \xi_{12} & \cdots & \xi_{1K} \\ \xi_{21} & \xi_{22} & \cdots & \xi_{2K} \\ \cdots & \cdots & \cdots & \cdots \\ \xi_{N1} & \xi_{N2} & \cdots & \xi_{NK} \end{pmatrix}$$

При этом старый набор должен приближённо восстанавливаться из нового линейным преобразованием:

$$\hat{Z}_{NxM} = Q_{NxK} W_{KxM}$$

Вопрос: а почему $\hat{Z}$, а не Z?

Зависимости между параметрами

Вопрос: как влияют на матрицу Z линейные зависимости между параметрами (столбцами)?
Ответ: они понижают её ранг.

Попытаемся заменить $(x_{i1}, x_{i2}, ..., x_{iM})$ на другой набор признаков меньшей длины $(\xi_{i1}, \xi_{i2}, ..., \xi_{iK}), K < M$:

$$Q = \begin{pmatrix} \xi_{11} & \xi_{12} & \cdots & \xi_{1K} \\ \xi_{21} & \xi_{22} & \cdots & \xi_{2K} \\ \cdots & \cdots & \cdots & \cdots \\ \xi_{N1} & \xi_{N2} & \cdots & \xi_{NK} \end{pmatrix}$$

При этом старый набор должен приближённо восстанавливаться из нового линейным преобразованием:

$$\hat{Z}_{NxM} = Q_{NxK} W_{KxM}$$

Вопрос: а почему $\hat{Z}$, а не Z?
Ответ: так как $K < M$, часть данных старой выборки будет утрачена и не может быть восстановлена.

Вспомогательное замечание

Для расчёта суммы квадратов элементов матрицы можно воспользоваться соотношением:

$$\|A\|^2 = tr(A^T A),$$

т.е. нужно перемножить транспонированную матрицу с самой собой и просуммировать диагональ. Пример для пояснения:

$$\begin{pmatrix} a & b & c \\ d & e & f \end{pmatrix} \begin{pmatrix} a & d \\ b & e \\ c & f \end{pmatrix} = \begin{matrix} a^2+b^2+c^2 & \dots \\ \dots & d^2+e^2+f^2 \end{matrix}$$

Задача минимизации

Попытаемся минимизировать ошибку восстановления старого набора по новому методом наименьших квадратов:

$$\varepsilon = \|E\| = \|\hat{Z} - Z\| = \|(QW - Z)\| = \sqrt{\sum_{i=0}^{N}\sum_{j=0}^{M}(\hat{x}_{ij} - x_{ij})^2}$$

Вопрос: как будут выглядеть необходимые условия минимума?

Задача минимизации

Попытаемся минимизировать ошибку восстановления старого набора по новому методом наименьших квадратов:

$$\varepsilon = \|E\| = \|\hat{Z} - Z\| = \|(QW - Z)\| = \sqrt{\sum_{i=0}^{N}\sum_{j=0}^{M}(\hat{x}_{ij} - x_{ij})^2}$$

Вопрос: как будут выглядеть необходимые условия минимума?

Ответ: $\frac{\partial E^T E}{\partial W} = 0$, $\frac{\partial E^T E}{\partial Q} = 0$. $E^T E = (QW - Z)^T (QW - Z)$ и:

$$\frac{\partial E^T E}{\partial W} = 2Q^T(QW - Z) = 0, \frac{\partial E^T E}{\partial Q} = 2(QW - Z)W^T = 0$$

Если считать матрицы Q и W невырожденными, то:

$$Q^T QW - Q^T Z = 0, QWW^T - ZW^T = 0$$

$$W = (Q^T Q)^{-1} Q^T Z, Q = ZW^T (WW^T)^{-1}$$

Задача минимизации (2)

$$W = (Q^T Q)^{-1} Q^T Z, Q = Z W^T (W W^T)^{-1}$$

Видно, что придётся обращать матрицы $Q^T Q$ и $W W^T$. Попытаемся как-то облегчить себе работу.

В матрице ошибок $E = QW - Z$ мы можем варьировать Q и W при условии, что их произведение останется постоянным. Это значит, что можно записать для любой невырожденной матрицы R_{KxK}:

$$QW = QRR^{-1}W = (QR)(R^{-1}W) = \check{Q}\check{W}$$

Попытаемся подобрать R так, чтобы полученные матрицы $\check{Q}^T\check{Q}$ и $\check{W}\check{W}^T$ стали диагональными.

Задача минимизации (3)

$\check{W}\check{W}^T$ — симметричная, поэтому найдётся невырожденная матрица S_{KxK}: $S\check{W}\check{W}^T S^T = I$.
Рассмотрим матрицу $S^{-1T}\check{Q}^T\check{Q}S^{-1}$. $\check{Q}^T\check{Q}$ симметрична, поэтому она будет также симметричной. Существует ортогональная матрица T_{KxK} такая, что матрица $T^{-1T}S^{-1T}\check{Q}^T\check{Q}S^{-1}T^{-1} = \Lambda$ — диагональная.
Применим для получения $\check{Q}$ и $\check{W}$ преобразование $R = TS$.
Тогда получим для прежних Q и W:

$$\check{Q} = QR \Rightarrow Q = \check{Q}R^{-1}$$

$$\check{W} = R^{-1}W \Rightarrow W = R\check{W}$$

Тогда:

$$WW^T = R\check{W}\check{W}^T R^T = TS\check{W}\check{W}^T S^T T^T = TIT^T = I$$

$$Q^T Q = (\check{Q}R^{-1})^T\check{Q}R^{-1} = R^{-1T}\check{Q}^T\check{Q}R^{-1} = T^{-1T}S^{-1T}\check{Q}^T\check{Q}S^{-1}T^{-1} = \Lambda$$

Задача минимизации (4)

Всё это значит, что существуют такие W и Q, что $WW^T = I$ и $Q^TQ = \Lambda$ — диагональны (их легко обратить) и при этом их произведение — решение нашей задачи минимизации.

Опять выпишем необходимые условия минимума:

$$W = (Q^TQ)^{-1}Q^TZ$$

$$Q = ZW^T(WW^T)^{-1}$$

Учтём полученные соотношения:

$$W = \Lambda^{-1}Q^TZ \Rightarrow \Lambda W = Q^TZ$$

$$Q = ZW^T$$

Подстановки

После подстановки второго выражения в первое получаем:

$$\Lambda W = (ZW^T)^T Z = WZ^T Z$$

Вывод: строки матрицы W — собственные векторы матрицы $Z^T Z$, соответствующие собственным числам, стоящим на диагонали Λ.
После подстановки первого выражения во второе аналогично получаем:

$$Q = Z(\Lambda^{-1} Q^T Z)^T = ZZ^T Q\Lambda^{-1T}$$

$$\Lambda^T Q = \Lambda Q = ZZ^T Q$$

Вывод: столбцы матрицы Q — собственные векторы матрицы ZZ^T, соответствующие тем же собственным числам.

Ошибка приближения

$$\varepsilon = \|E\| = \|\hat{Z} - Z\| = \|(QW - Z)\| = tr(QW - Z)^T(QW - Z) =$$

$$= tr(W^TQ^T - Z^T)(QW - Z) = tr(W^TQ^TQW - Z^TQW - W^TQ^TZ + Z^TZ) =$$

(учтём, что $Q^TQ = \Lambda$ и $Q^TZ = \Lambda W$)

$$= tr(W^T\Lambda W - Z^TQW - W^T\Lambda W + Z^TZ) = tr(Z^TZ - Z^TQW) =$$

(учтём, что $Z^TQ = (\Lambda W)^T$)

$$= tr(Z^TZ - W^T\Lambda^TW) = \sum_{i=1}^{M}\lambda_i - \sum_{i=1}^{K}\lambda_i = \sum_{i=K+1}^{M}\lambda_i$$

Вывод: ошибка приближения равна сумме собственных чисел матрицы Z^TZ, не попавших в Λ. Если использовать для Λ K максимальных собственных чисел, ошибка будет минимальной.

Применения метода

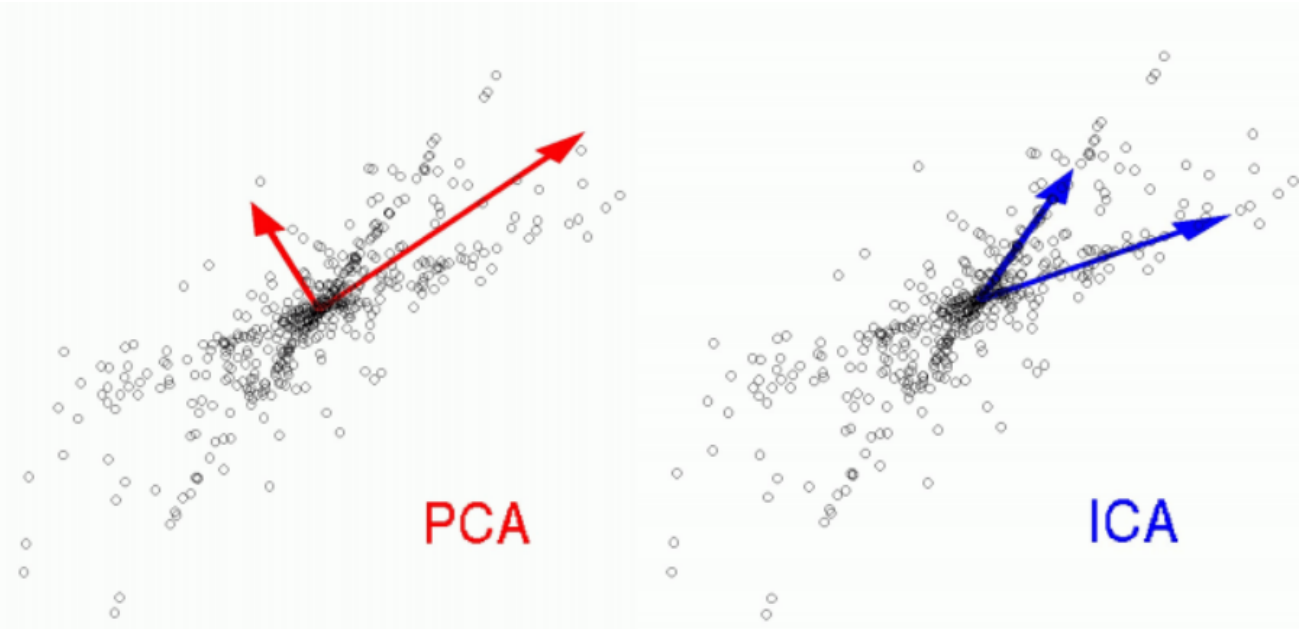
- Снижение количества параметров: можно, например, провести PCA, отсортировать новые параметры по уменьшению λ и отбросить те, сумма λ которых слабо влияет на результат.
- двумерная визуализация многомерных данных для обнаружения скрытой структуры: сгущений, разрежений, потенциальных компонентов смесей, кластеров и т.п.

В принципе, сам этот метод относится к **обучению без учителя**: данные преобразуются только по параметрам, без заранее данных правильных ответов / номеров классов / их требуемого количества / значений функций.

Выделение независимых компонентов

Выделение главных компонентов отбирает на каждом шаге направление, в котором выборочная дисперсия максимальна (т.е. данные изменяются сильнее всего).

Иногда требуется немного другое — отделить друг от друга разные независимые составляющие выборки. Тогда используется выделение независимых компонентов (Independent Component Analysis, ICA):



Пример задачи: шумное помещение

Пусть в одном помещении много разных, независимых друг от друга источников звука (неподвижных). Также в ней находится несколько неподвижных микрофонов, записывающих звук в отдельных точках. Можно ли, проанализировав записи со смесями звуков, найти исходные звуки от каждого источника?

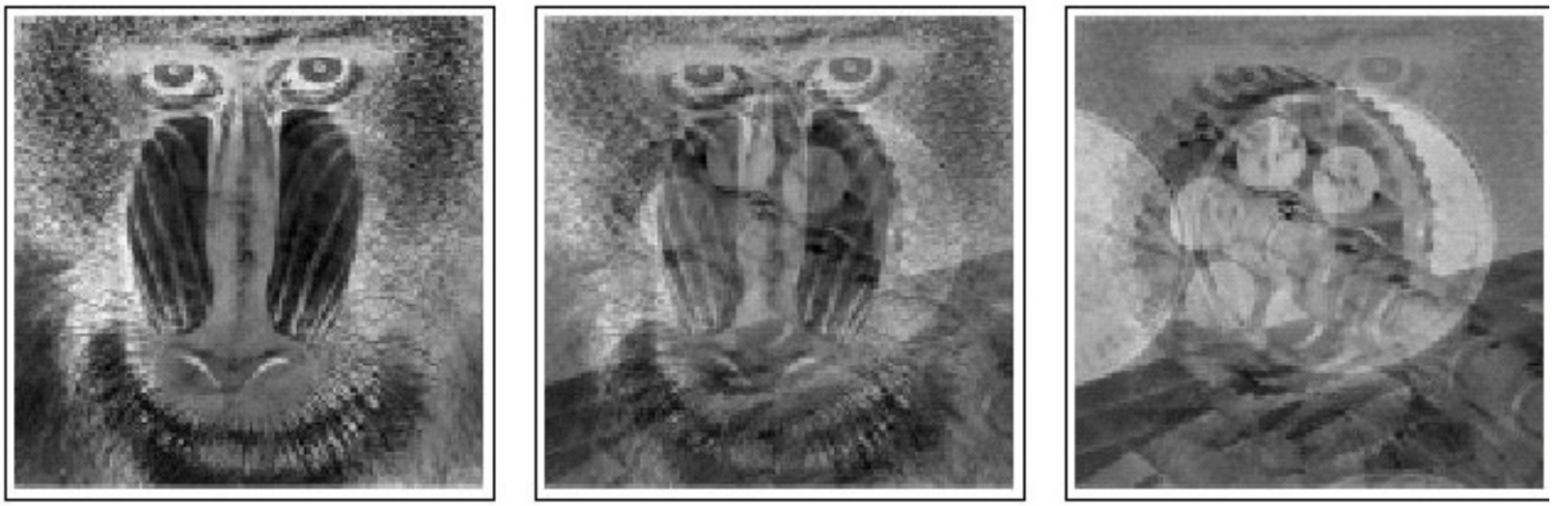
Пример: если источников звука два — $X_1(t)$ и $X_2(t)$, то микрофоны могут записать, например, $Z_1(t) = 0.1X_1(t) + 0.6X_2(t)$ и $Z_2(t) = 0.75X_1(t) + 0.2X_2(t)$.

Конкретнее: каждый звуковой сигнал можно закодировать временным рядом $X_i = [x_{i1}, x_{i2}, ..., x_{iN}], i = 1..M$; микрофоны записывают линейные комбинации данных сигналов. Если собрать все столбцы X_i в матрицу X_{NxM}, то матрица результатов записи Z вычисляется как: $Z = AX$, где A — матрица постоянных коэффициентов (матрица смешивания). Известна Z, требуется найти X.

Пример задачи: шумное помещение (2)

Наглядный пример из первоисточника [Langlois, D., Chartier, S. and Gosselin, D. An Introduction to Independent Component Analysis: InfoMax and FastICA Algorithm. Tutorials in Quantitative Methods for Psychology, v. 6, 2010, pp.31-38.]

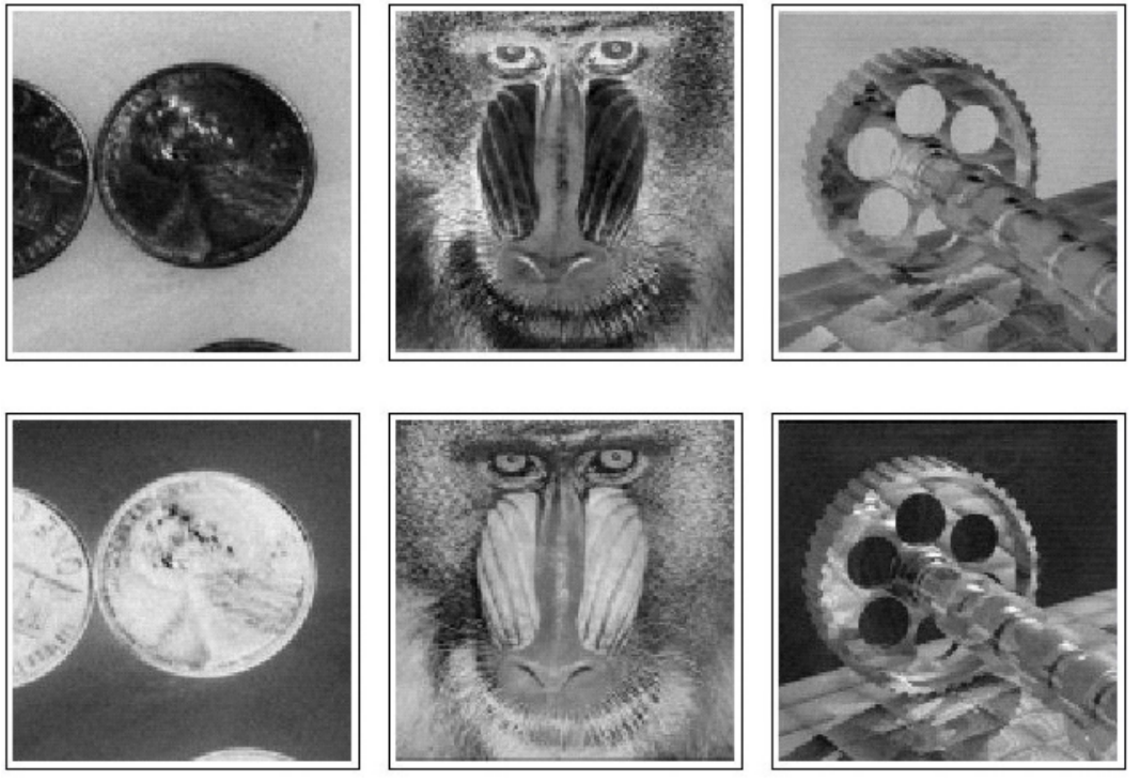
Три независимых изображения смешали с неизвестными коэффициентами:



Требуется найти оригинальные изображения.

Пример задачи: шумное помещение (3)

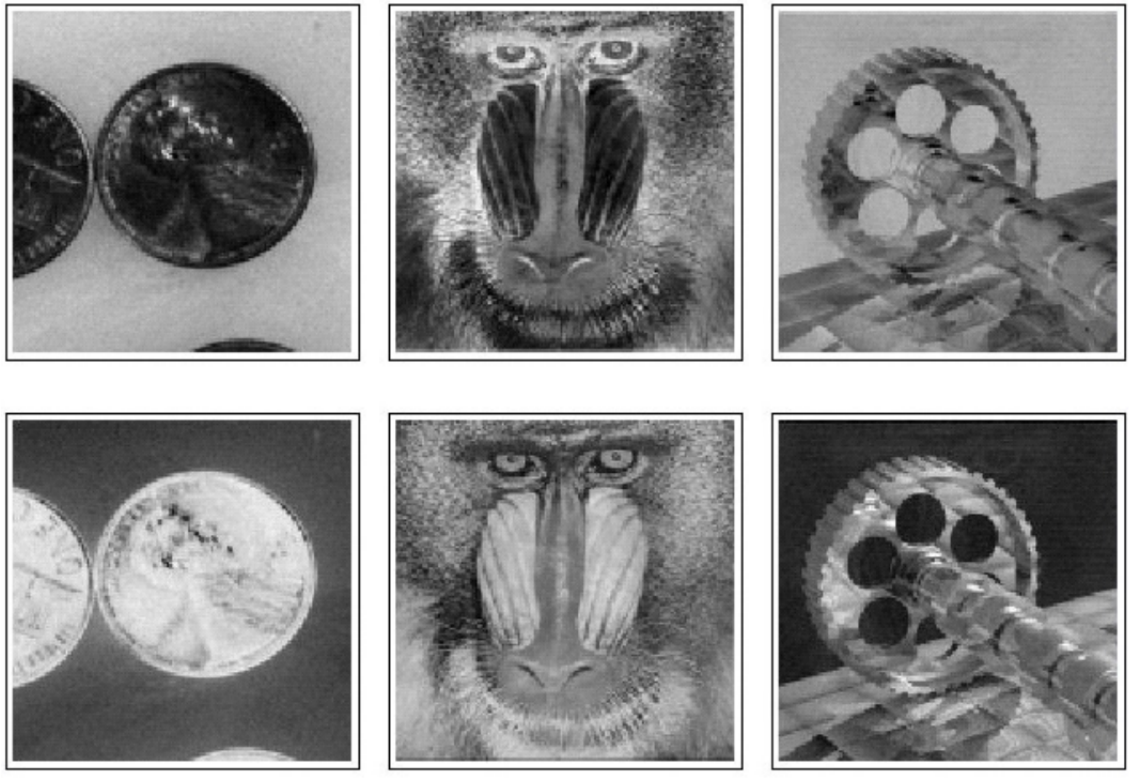
Решение будет не единственным:



Вопрос: почему?

Пример задачи: шумное помещение (3)

Решение будет не единственным:



Вопрос: почему?

Ответ: потому что компоненты определены с точностью до постоянного множителя.

Основные подходы

Единственно верного алгоритма ICA нет, есть несколько подходов с разными оптимизируемыми показателями:

- максимизация негауссовости: если данные распределены нормально, то они, скорее всего, получены сочетанием множества разных сигналов, а нам нужны только отдельные сигналы;
- минимизация взаимной информации: если разные компоненты не влияют друг на друга, они, скорее всего, порождены разными сигналами;
- максимизация правдоподобия того, что именно проверяемая матрица смешивания породила выборку.

Рассмотрим основные этапы без строгих доказательств.

Предобработка

$$Z = AX$$

Независимо от дальнейших действий всегда выполняются:

- центрирование: из каждого сигнала (столбца Z) вычитается его среднее значение;
- декорреляция: каждый сигнал (столбец Z) делается некоррелированным со всеми другими, например выделением главных компонент; значения в Z заменяются на эти некоррелированные сигналы;
- масштабирование: каждый сигнал домножается на постоянный коэффициент, чтобы его дисперсия стала равна 1.

Подход 1: негауссовость через эксцесс

Для случайной величины можно определить **нормализованный эксцесс** (normalized kurtosis):

$$\hat{K}(X) = \frac{E[X^4]}{E[X^2]^2} - 3,$$

где $E[Y]$ — среднее значение Y по выборке. Для гауссова распределения $\hat{K} = 0$; чем сильнее распределение отличается от нормального, тем больше $\hat{K}$ отличается от нуля.

Для эксцесса выполняются свойства:

$$\hat{K}(Y_1 + Y_2) = \hat{K}(Y_1) + \hat{K}(Y_2), \hat{K}(\alpha Y) = \alpha^4 \hat{K}(Y)$$

Если, например, смешаны два компонента, то $\hat{K}(Z) = \hat{K}(aX_1 + bX_2) = a^4\hat{K}(X_1) + b^4\hat{K}(X_2)$. Если дисперсии Z, X_1 и X_2 единичны, то $1 = E[Z^2] = a^2E[X_1^2] + b^2E[X_2^2] = a^2 + b^2 = 1$, т.е. нужно найти максимальное значение выражения $a^4\hat{K}(X_1) + b^4\hat{K}(X_2)$ на круге $a^2 + b^2 = 1$. Это можно сделать градиентным спуском.

Подход 2: негауссовость через отрицательную энтропию

Энтропия случайной переменной u, которая распределена с плотностью $p(u)$ на своей области определения $\mathbb{U}$, равна

$$H(u) = -\int\limits_{u \in \mathbb{U}} p(u) \log p(u) du$$

Назовём отрицательной энтропией разность между энтропией случайной величины u и энтропией гауссовой случайной величины u_{norm} с такой же матрицей ковариации:

$$J(u) = H(u_{norm}) - H(u)$$

Так как энтропия нормально распределённой величины максимальна при заданной матрице ковариации, $J(Z)$ всегда положительна. Если её максимизировать, можно получить независимые компоненты.

Подход 2: негауссовость через отрицательную энтропию (2)

На практике считать энтропию по её определению сложно; чаще используют приблизительные формулы, например

$$J(u) \approx \sum_{i=1}^{N} k_i (E[G_i(u)] - E[G_i(u_{norm})])^2,$$

(Hyvarinen, 1998), где N — число независимых компонентов, G_i — положительные функции, k_i — положительные коэффициенты. При $G(u) = u^4$ получаем ту же задачу, что для эксцесса.

Типичные варианты таких функций:

$$G(u) = -e^{-u^2/2}$$

$$G(u) = \frac{1}{a} \log \cosh(au)$$

Подход 3: минимизация взаимной информации

Взаимная информация между случайными переменными $x_i, i = 1..N$, составляющими вместе матрицу Z:

$$I(x_1, ..., x_N) = \sum_{i=1}^{N} H(x_i) - H(Z)$$

Можно показать, что этот вариант легко сводится к предыдущему, если вместо Z подставить WY, где Y — матрица независимых случайных переменных, W — искомое преобразование.

Применяемые алгоритмы

Полезно будет ознакомиться с используемыми на практике алгоритмами, так как они включают множество эвристических стадий:

- FastICA;
- Projection pursuit;
- Infomax.

Источники:

1. Hyvärinen A, Oja E. Independent component analysis: algorithms and applications. Neural Netw. 2000 May-Jun; v 13(4-5): pp.411–430. DOI: 10.1016/s0893-6080(00)00026-5.
2. Langlois D., Chartier S., Gosselin D. An Introduction to Independent Component Analysis: InfoMax and FastICA algorithms. Tutorials in Quantitative Methods for Psychology, v. 6, 2010. DOI: 10.20982/tqmp.06.1.p031.

FastICA

A. Hyvärinen, E. Oja, 1999

Общая идея: оптимизировать негэнтропию, записанную в виде

$$J(u) \approx (E[G(u)] - E[G(u_{norm})])^2$$

Эта формула отличается от приведённой выше тем, что все G_i предполагаются равными.

Решение ищется градиентным спуском. Для одной компоненты выбирается начальное значение её коэффициентов w, после чего проводится спуск:

$$w_{new} = xG'(w^T x) - G''(w^T x)w$$

После спуска по каждой компоненте отдельно обеспечивают их ортогональность, например процессом Грама-Шмидта.

Тема 7

Кластеризация

Задача обработки текста

Кластеризация

Постановка задачи
Графовые методы
Метрические методы
Статистические методы
Иерархические методы
Критерии качества кластеризации

Задача кластеризации

Задача: пусть имеется выборка Z, состоящая из описаний объектов. Требуется разделить её на группы (**кластеры**), в каждой из которых описания объектов схожи друг с другом, а в разных группах — различны.

Уточнения:

- в некоторых методах любой объект обязательно включается в какой-либо кластер, в других — нет;
- иногда каждый объект включают не более чем в один кластер, иногда кластеры могут образовывать иерархию: крупные кластеры включают в себя более мелкие, при этом сходство внутри мелких кластеров ярче выражено, чем внутри крупных;
- иногда о принадлежности объекта к кластеру нет никаких данных (получается задача обучения без учителя), иногда для некоторых объектов кластер известен (получается задача частичного обучения).

Цели кластеризации

Вопрос: зачем это нужно?

Цели кластеризации

Вопрос: зачем это нужно?
Ответ:

- оценить структуру данных (например, для дальнейшего применения смеси);
- кратко описать структуру данных (например, чтобы убрать из выборки однотипные прецеденты);
- выявить выбросы и нетипичные объекты, если метод позволяет каким-либо прецедентам не попадать ни в какой класс.

Специфика кластеризации

Вопрос: а почему в определении было сказано про некое абстрактное «сходство описаний», а не более конкретно — «норма вектора разности», «расстояние», «вероятность равенства»?

Специфика кластеризации

Вопрос: а почему в определении было сказано про некое абстрактное «сходство описаний», а не более конкретно — «норма вектора разности», «расстояние», «вероятность равенства»?
Ответ: потому что часто нужно кластеризовать объекты, для которых вводить обычно использовавшиеся нами представления вроде набора конечного числа параметров $\{x_1, x_2, ..., x_K\}$ не получается.

Вопрос: можно ли придумать какие-нибудь объекты такого рода (плохо описываемые конечным, заранее известным числом параметров)?

Специфика кластеризации

Вопрос: а почему в определении было сказано про некое абстрактное «сходство описаний», а не более конкретно — «норма вектора разности», «расстояние», «вероятность равенства»?
Ответ: потому что часто нужно кластеризовать объекты, для которых вводить обычно использовавшиеся нами представления вроде набора конечного числа параметров $\{x_1, x_2, ..., x_K\}$ не получается.

Вопрос: можно ли придумать какие-нибудь объекты такого рода (плохо описываемые конечным, заранее известным числом параметров)?
Ответ: цепи ДНК, белки, тексты.

Некорректность задачи

Понятно, что в произвольном случае разделить объекты объективно правильно не получится; всегда можно подобрать сомнительные случаи, когда не будет понятно, какой вариант решения лучше.

Из-за этого разные подходы постоянно сочетают и параметризуют, пытаясь построить хорошее решение для конкретной задачи. Соответствие каким-либо критериям оптимальности часто игнорируют. Соответственно, методы очень разнообразны и скорее являются рекомендациями, чем точными алгоритмами.

Группы методов

По представлению данных:

- Графовые: считаем, что прецеденты — вершины графа, а расстояния между ними — веса дуг; кластеры — части графа с высокой связностью. Они же — метрические: считаем, что даны расстояния между вершинами; кластеры — группы, внутри которых расстояния маленькие, а между ними — большие;
- статистические: считаем, что прецеденты сгенерированы по некоей функции распределения, и пытаемся приблизить её смесью, каждый компонент которой — отдельный кластер;
- иерархические: считаем, что прецеденты должны объединяться во множество вложенных кластеров по критерию близости.

По охвату данных: полные (в кластеры входят все прецеденты) / неполные (могут остаться изолированные прецеденты), плоские / иерархические (создаются ли вложенные кластеры или нет);

Графовые методы

Прецеденты — вершины полносвязного ненаправленного графа, расстояния между ними — веса дуг. Надо выделить части, в которых в основном короткие дуги.

Подход 1. Выделение связных компонент.
Будем считать, что если расстояние между вершинами не больше некоторого заданного порота T, они входят в один кластер. Удаляем всё дуги с длиной больше T и находим связные компоненты в том, что осталось.
Вопрос: как найти в графе связные компоненты?

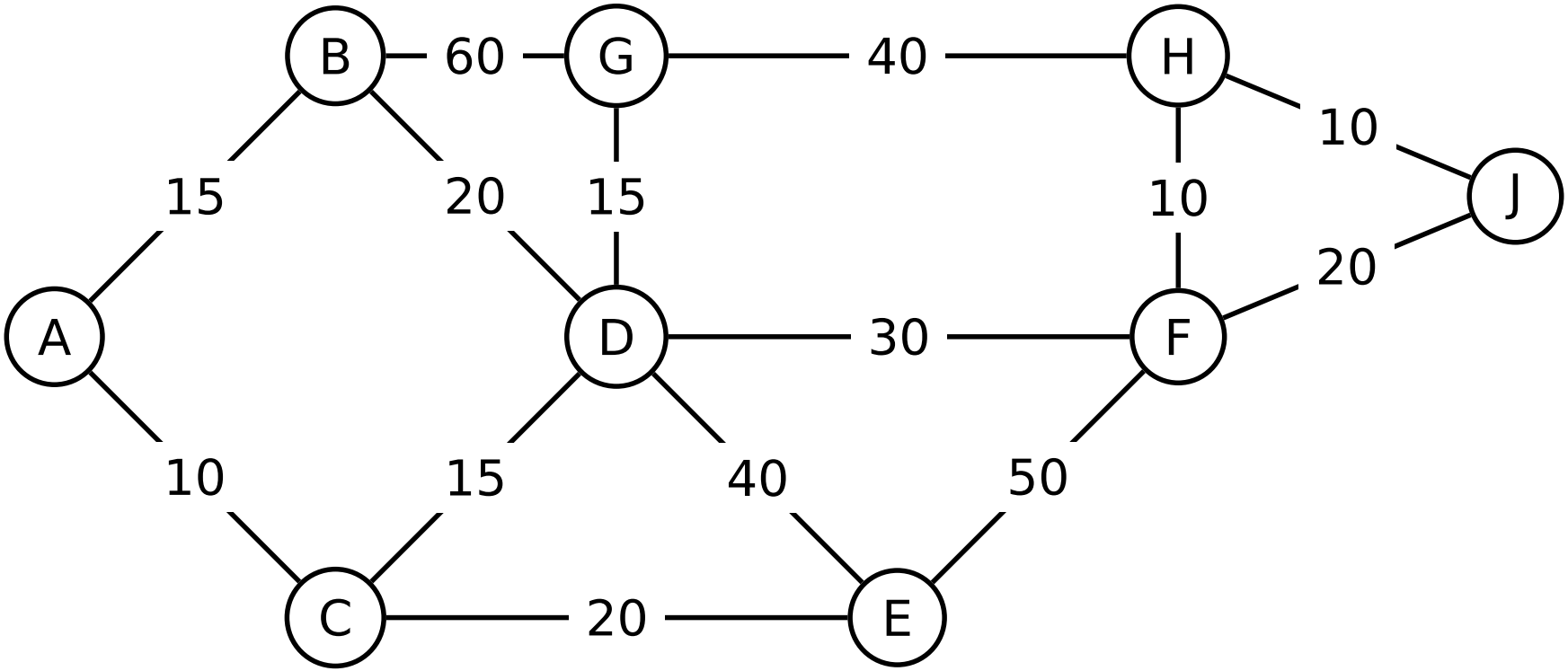
Графовые методы

Прецеденты — вершины полносвязного ненаправленного графа, расстояния между ними — веса дуг. Надо выделить части, в которых в основном короткие дуги.

Подход 1. Выделение связных компонент.
Будем считать, что если расстояние между вершинами не больше некоторого заданного порота T, они входят в один кластер. Удаляем всё дуги с длиной больше T и находим связные компоненты в том, что осталось.
Вопрос: как найти в графе связные компоненты?
Ответ: каким-либо обходом (поиском в ширину или поиском в глубину).

Пример графового метода

Рассмотрим граф такого вида:

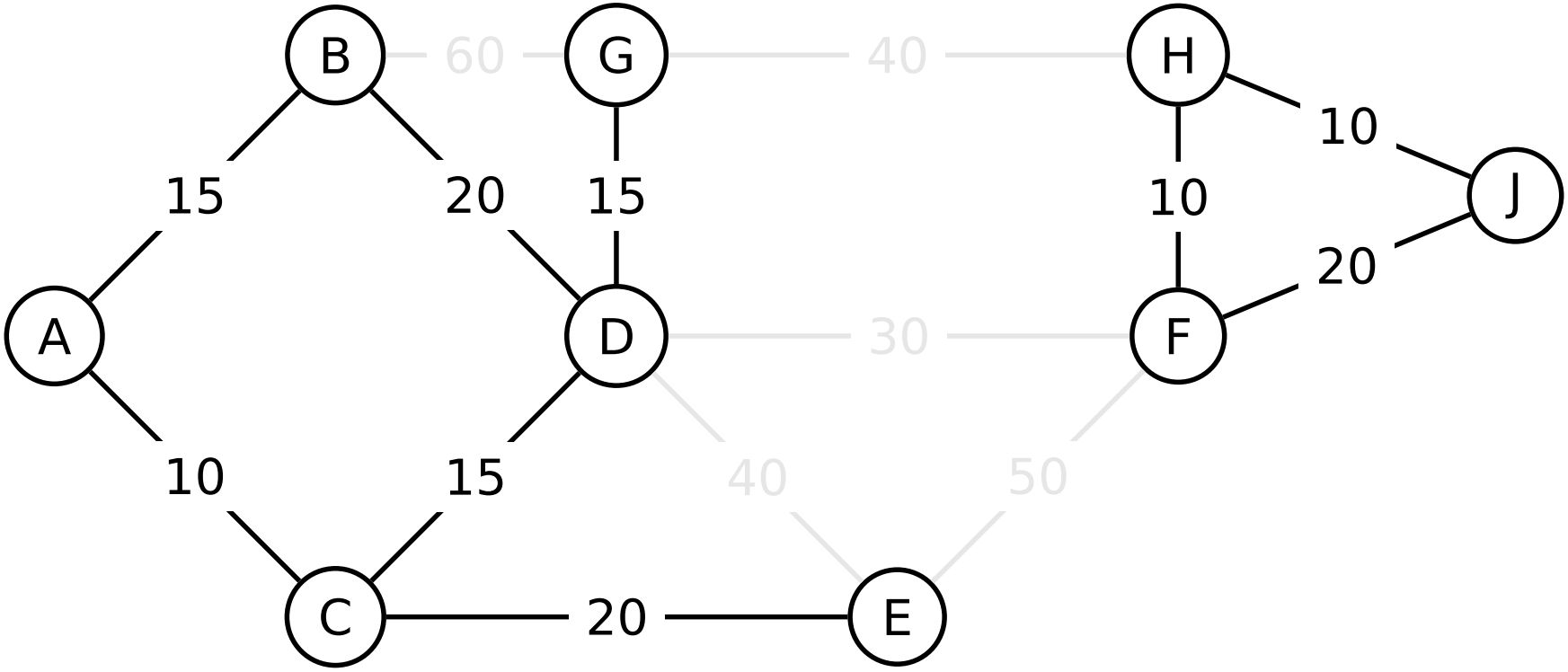


Будем считать, что все связи, которые не указаны, имеют очень большую длину (заведомо намного больше, чем любая из указанных или их сумма).

Зададимся порогом T и разделим граф на связные части по нему.

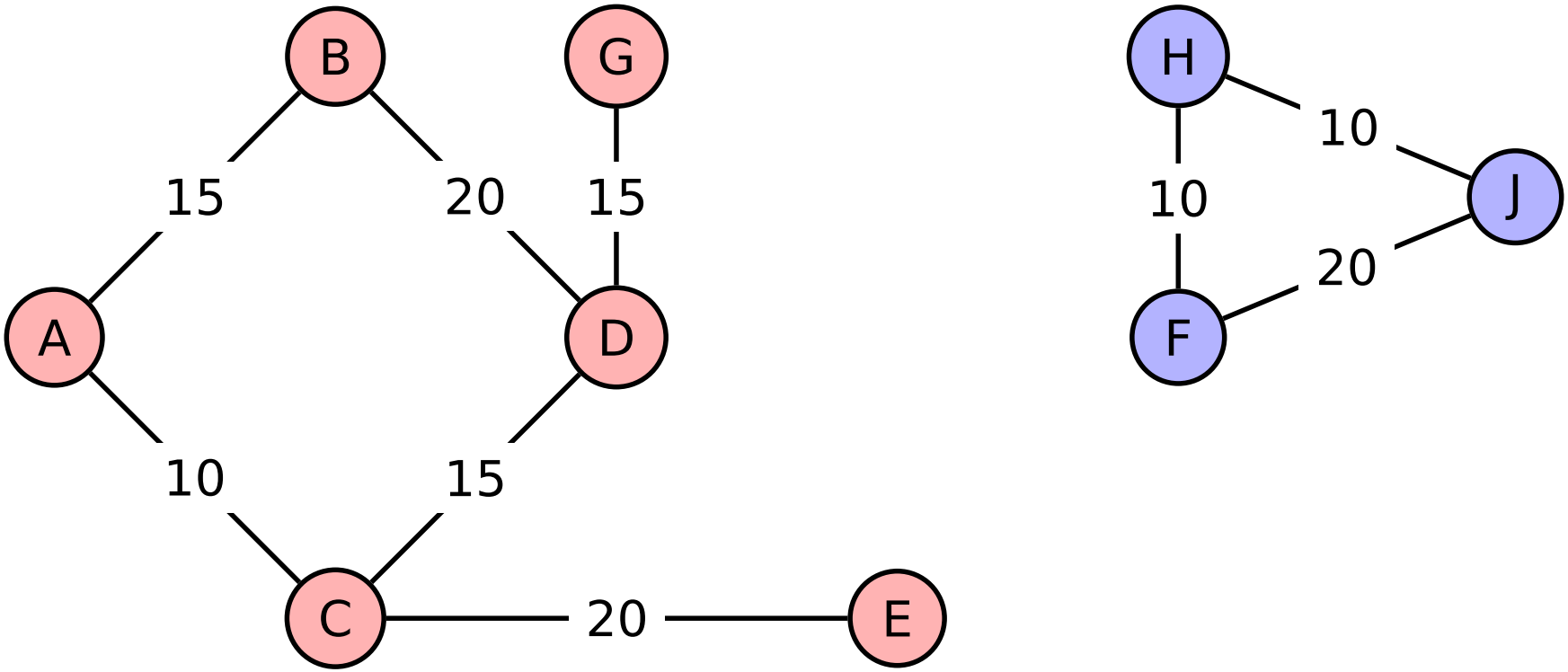
Пример графового метода (2)

Пусть порог T равен 25. Отбросим все связи с длиной больше T



Пример графового метода (2)

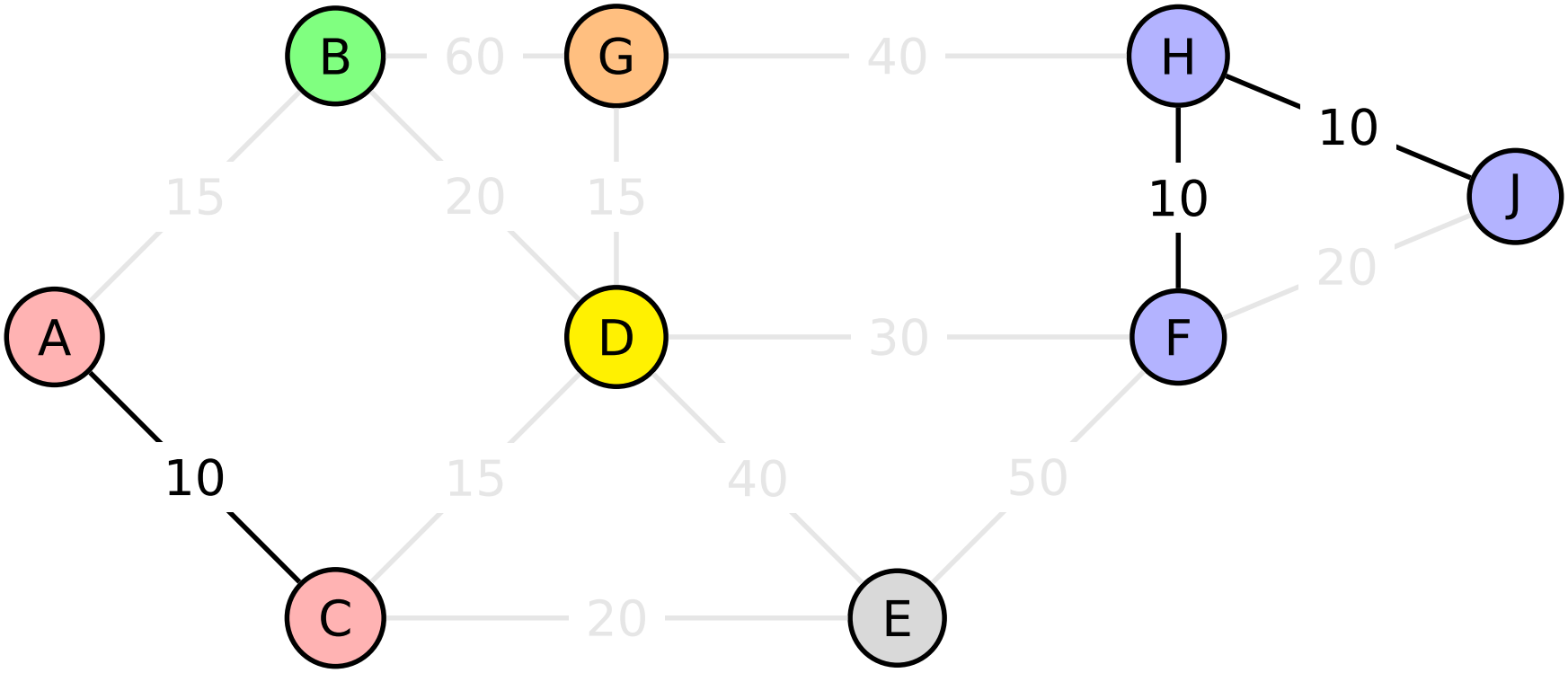
Пусть порог T равен 25. Отбросим все связи с длиной больше T



Видно, что граф распадается на связные компоненты, внутри которых все элементы достаточно схожи. При этом не обязательно каждый похож на каждого: чтобы два узла были в одном кластере, требуется какая-либо последовательность связей, а не прямая связь

Возможность построения иерархии

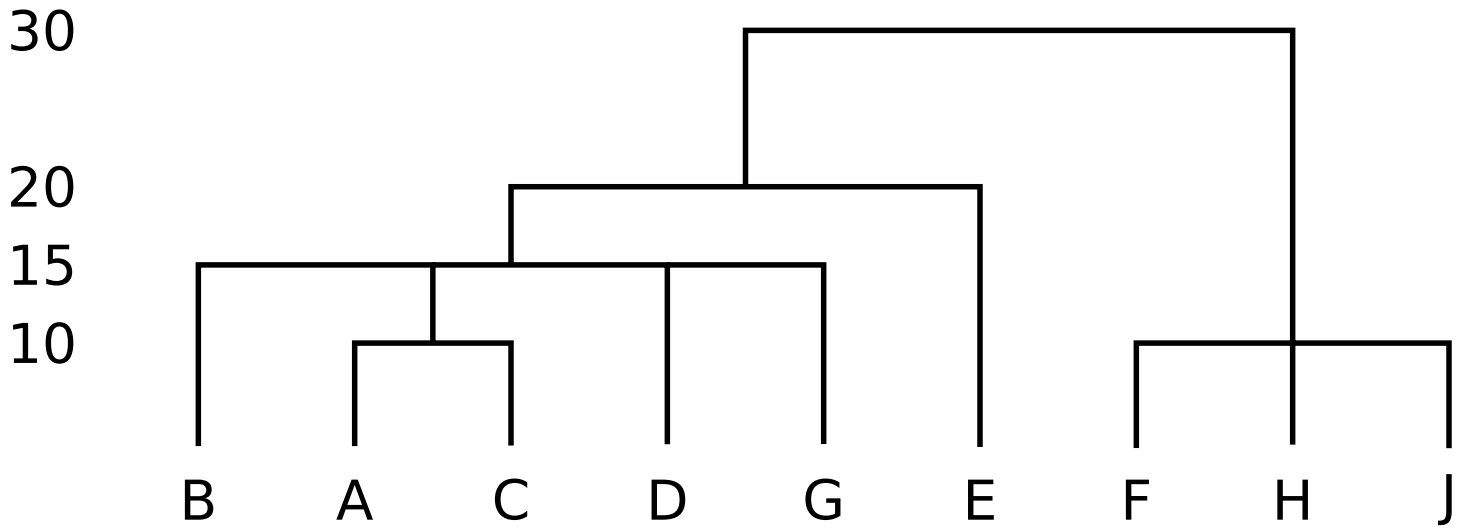
Если порог будет меньше, компонентов будет больше.
Например, при пороге 12:



Благодаря этому можно отследить зависимость между связностью и порогом.

Дендрограмма

Изобразим связи в виде графика: высота горизонтальной линии, связывающей группы, соответствует близости кластеров.



Например, самая верхняя горизонтальная связь проведена на высоте 30, потому что это минимальное расстояние между двумя самыми большими кластерами.

Этот график называется **дендрограмма**. По нему можно определить состав кластеров для любого порога, отбросив все связы выше него.

Итоги по связным компонентам

Достоинства: простота, понятность, возможность построения иерархии.

Недостатки: неясно, откуда брать T; плохо обрабатывает разреженные облака и узкие перемычки; предсказать число кластеров невозможно; границы между ними часто бывают очень прихотливыми.

Минимальное остовное дерево

Подход 2. Для любого связного графа можно построить связный ациклический подграф, включающий в себя все вершины.

Вопрос: как это сделать?

Минимальное остовное дерево

Подход 2. Для любого связного графа можно построить связный ациклический подграф, включающий в себя все вершины.
Вопрос: как это сделать?
Ответ: соединить любые две вершины, потом любую из несоединённых с любой соединённой и т.д., пока все вершины не будут соединены. Результат называется остовным деревом.

Вопрос: как его строить, чтобы сумма длин дуг была минимальна?

Минимальное остовное дерево

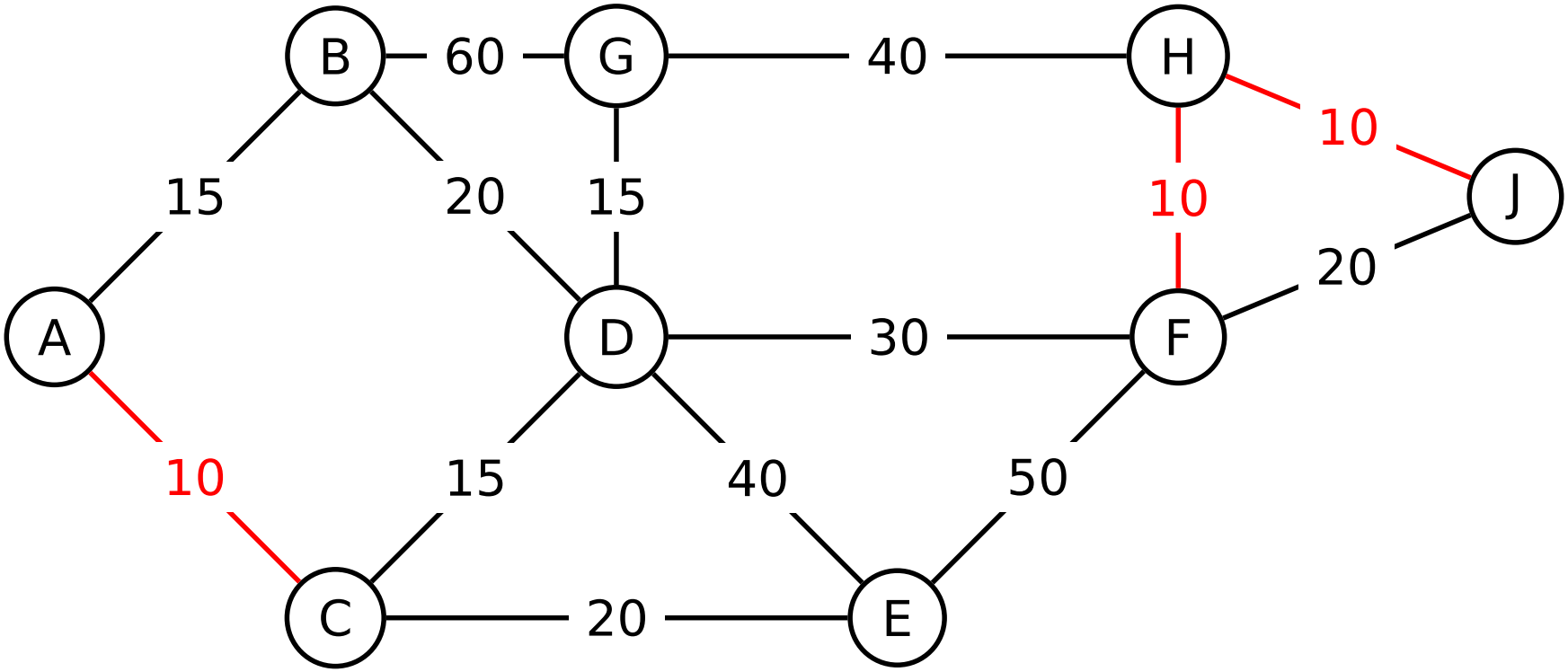
Подход 2. Для любого связного графа можно построить связный ациклический подграф, включающий в себя все вершины.
Вопрос: как это сделать?
Ответ: соединить любые две вершины, потом любую из несоединённых с любой соединённой и т.д., пока все вершины не будут соединены. Результат называется остовным деревом.

Вопрос: как его строить, чтобы сумма длин дуг была минимальна?
Ответ: действуем по жадному алгоритму. Включим в него сначала самую короткую дугу, потом самую короткую из тех, что соединяют любую невключенную вершину с любой включенной, и т.д., пока все вершины в него не попадут. Результат будет называться **минимальным остовным деревом**.

Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

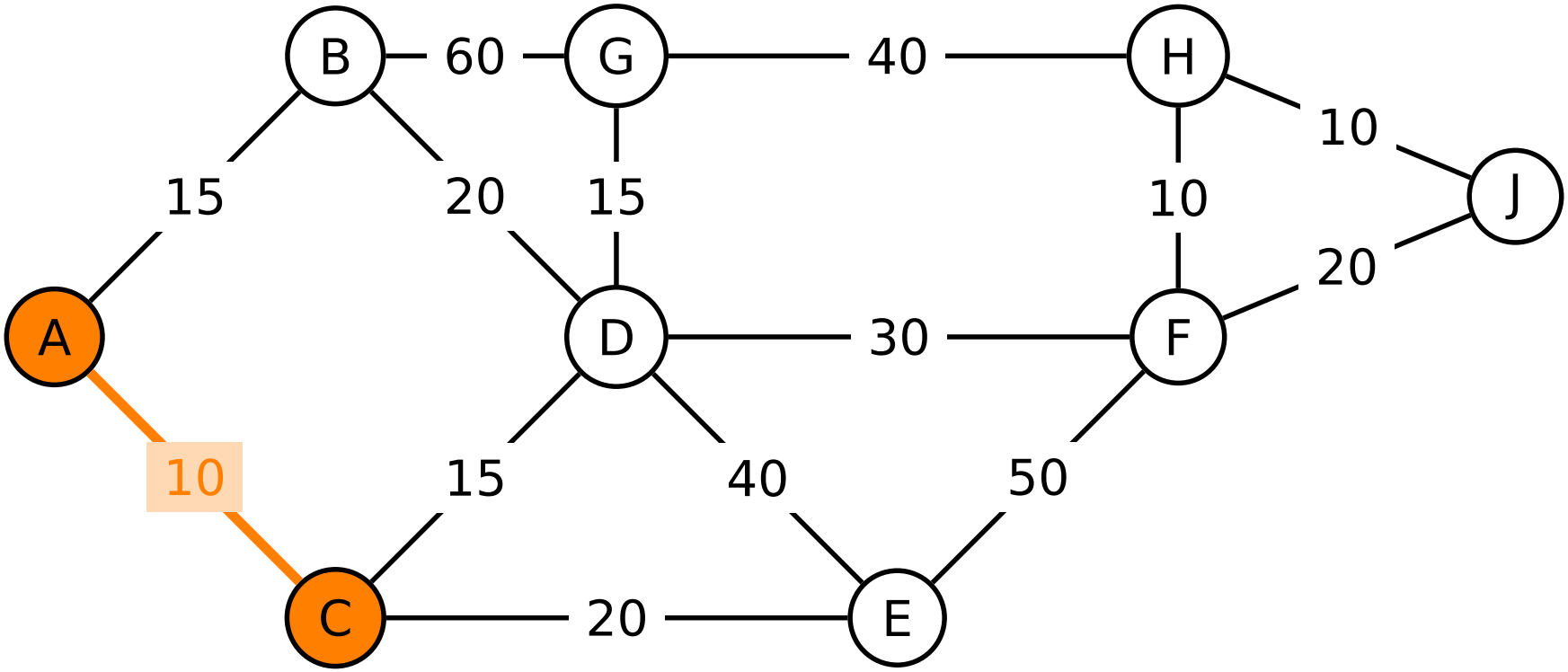
Найдём самые короткие связи из имеющихся и обозначим их красным



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

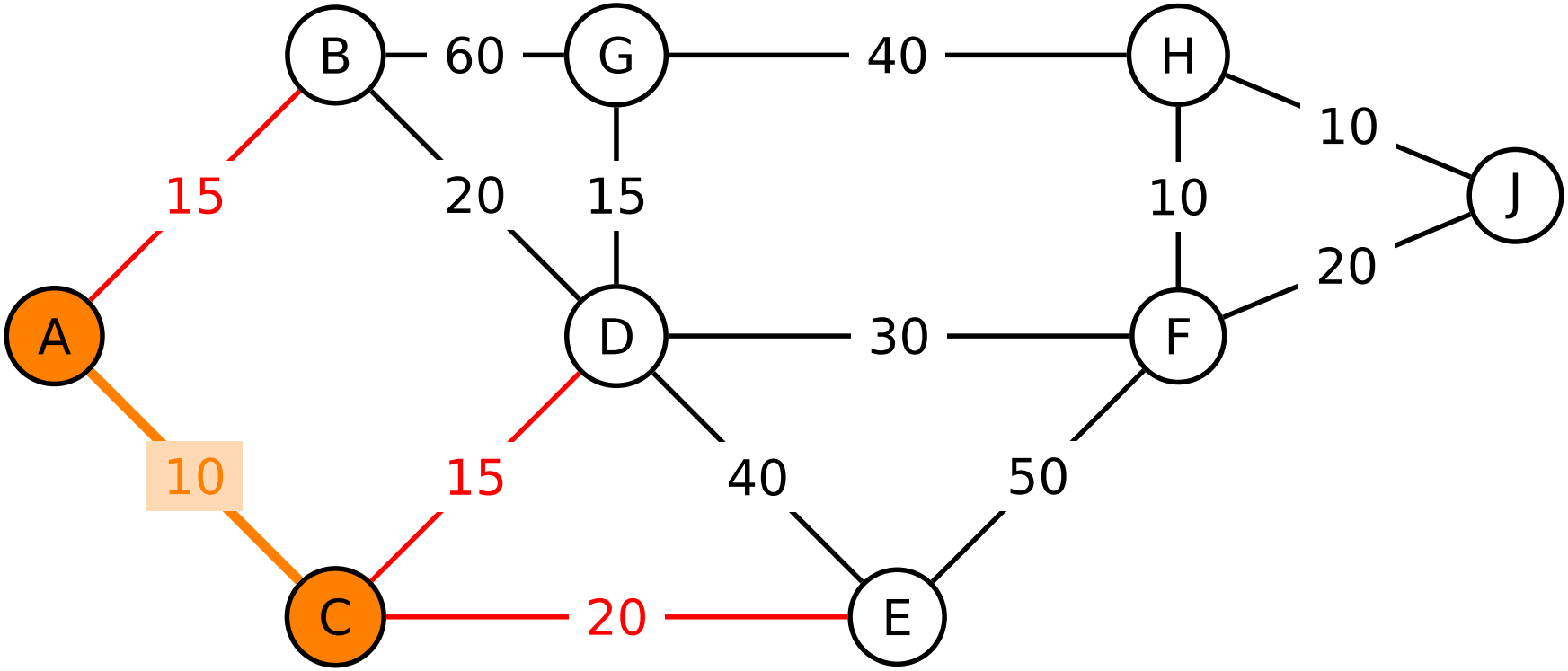
Случайно выберем одну из них и добавим её и соединённые ею вершины в дерево. Пусть это будет дуга AC



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

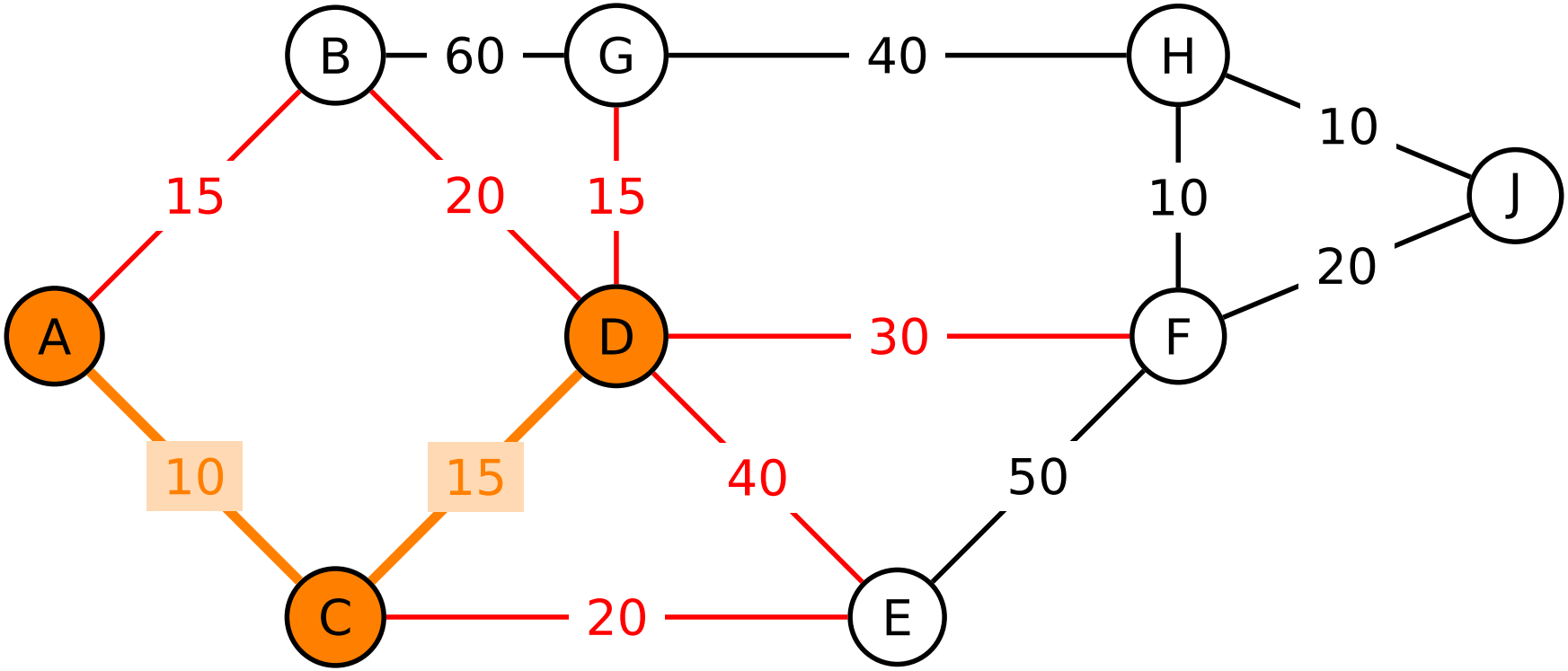
Найдём самую короткую из дуг, связывающих построенную часть дерева и ещё не охваченные вершины. Варианты: CE, CD, AB



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

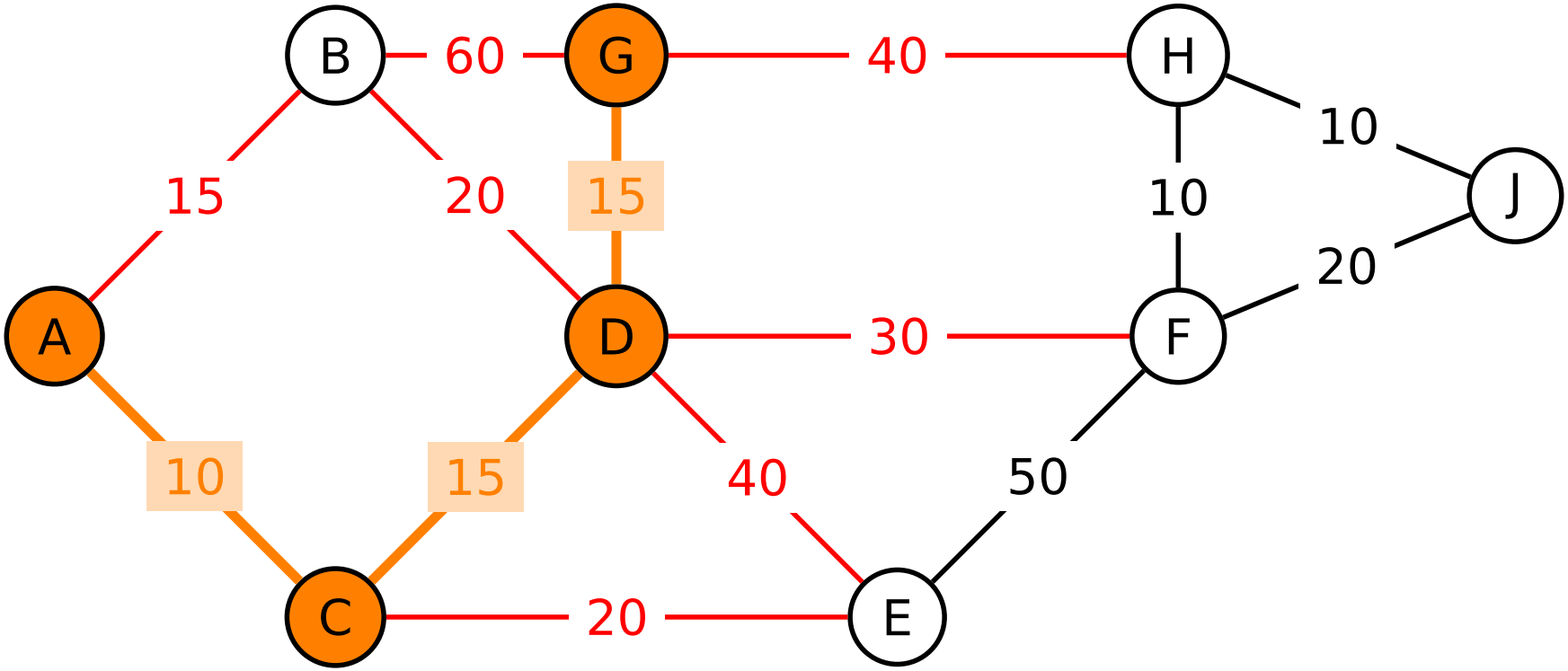
Выберем какую-нибудь из кратчайших дуг и добавим её к дереву. Снова находим варианты дуг, которые соединяют вершины дерева и неохваченные вершины



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

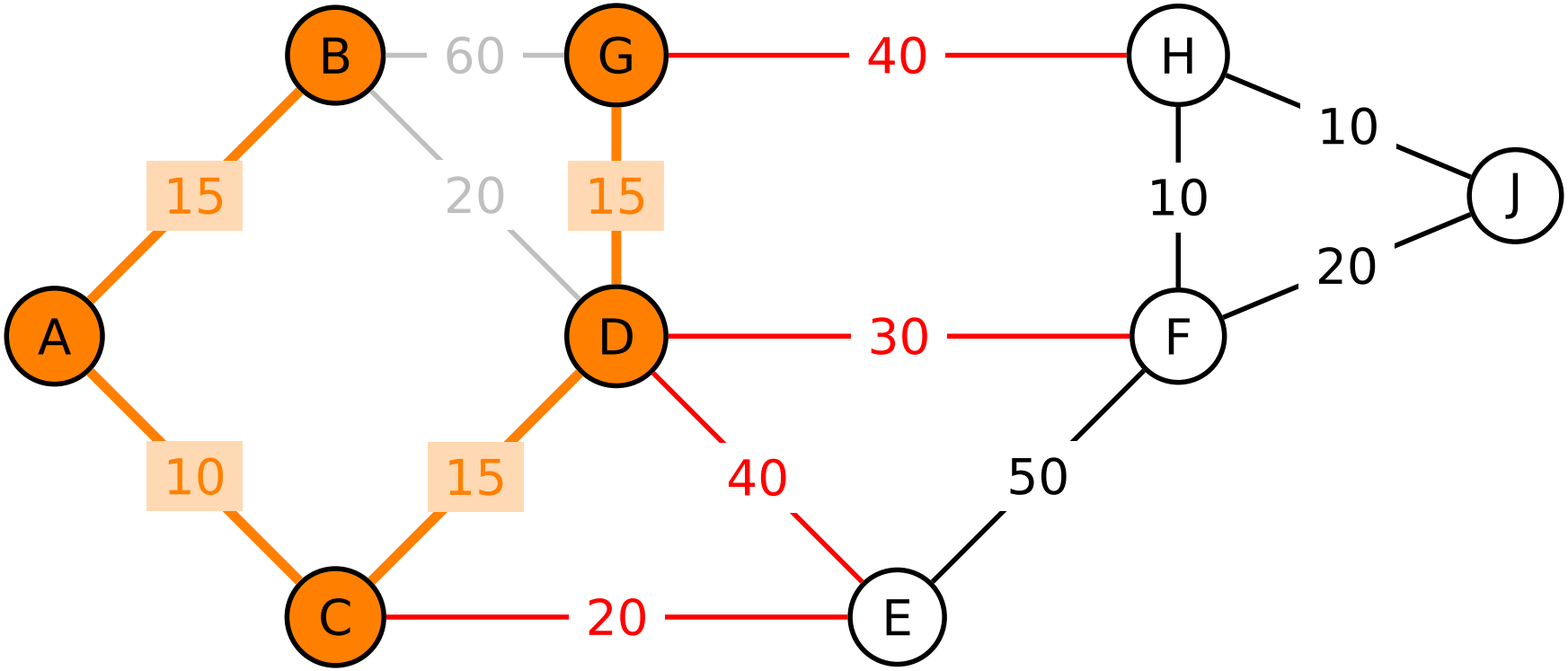
Выберем какую-нибудь из кратчайших дуг и добавим её к дереву. Снова находим варианты дуг, которые соединяют вершины дерева и неохваченные вершины



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

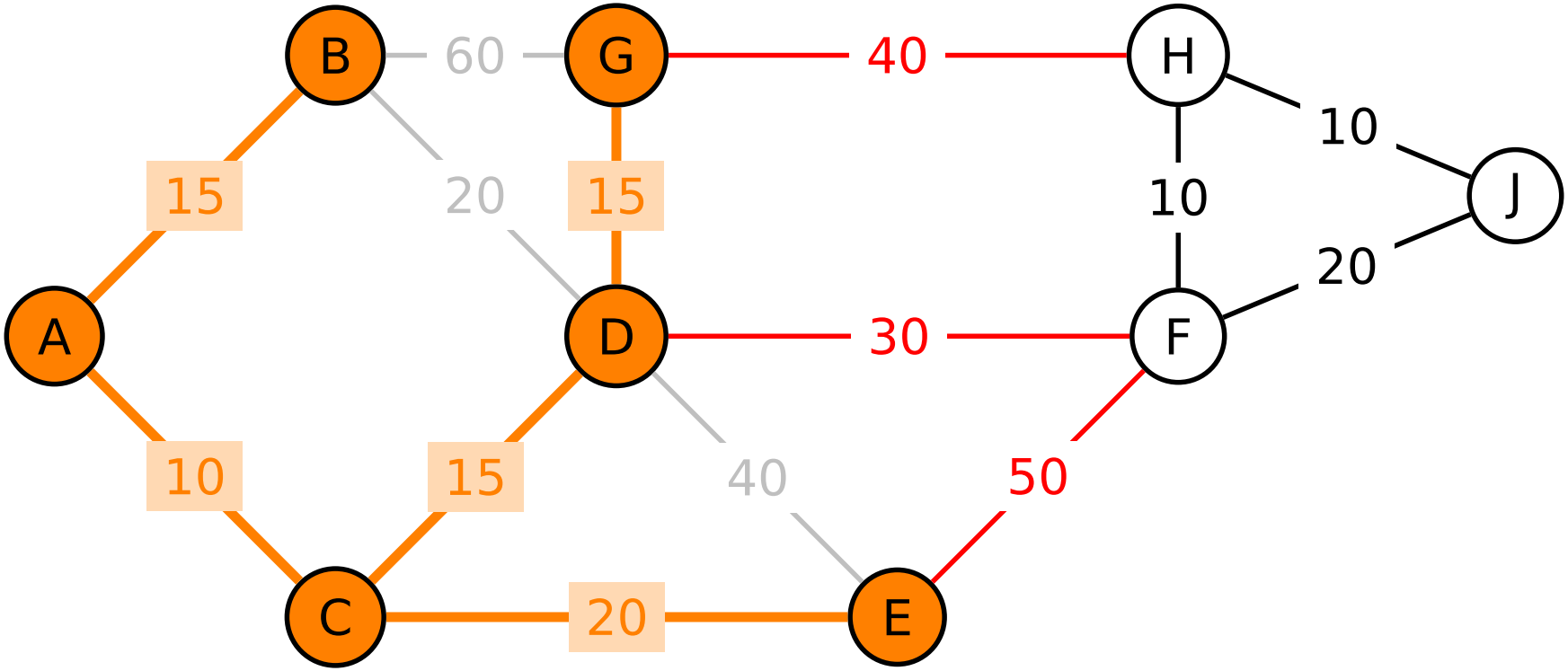
Выберем какую-нибудь из кратчайших дуг и добавим её к дереву. Снова находим варианты дуг, которые соединяют вершины дерева и неохваченные вершины



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

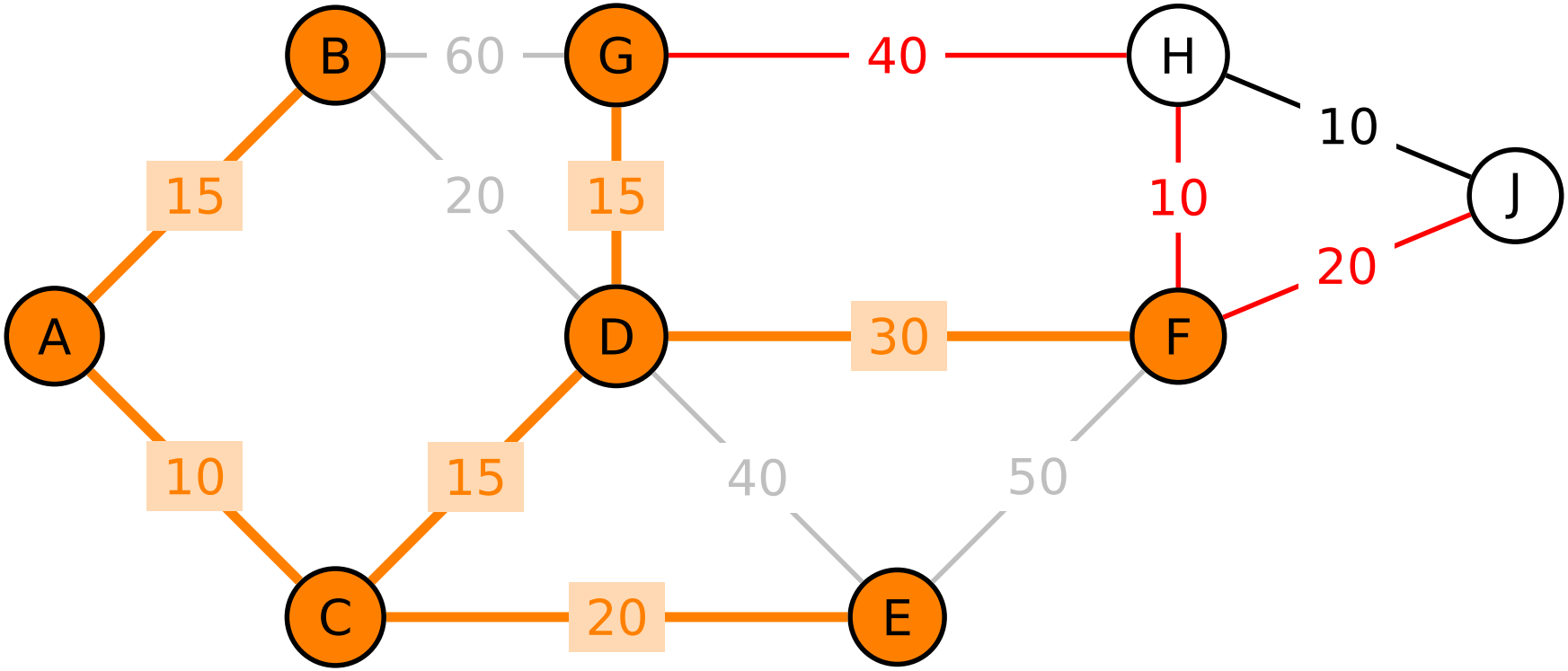
Выберем какую-нибудь из кратчайших дуг и добавим её к дереву. Снова находим варианты дуг, которые соединяют вершины дерева и неохваченные вершины



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

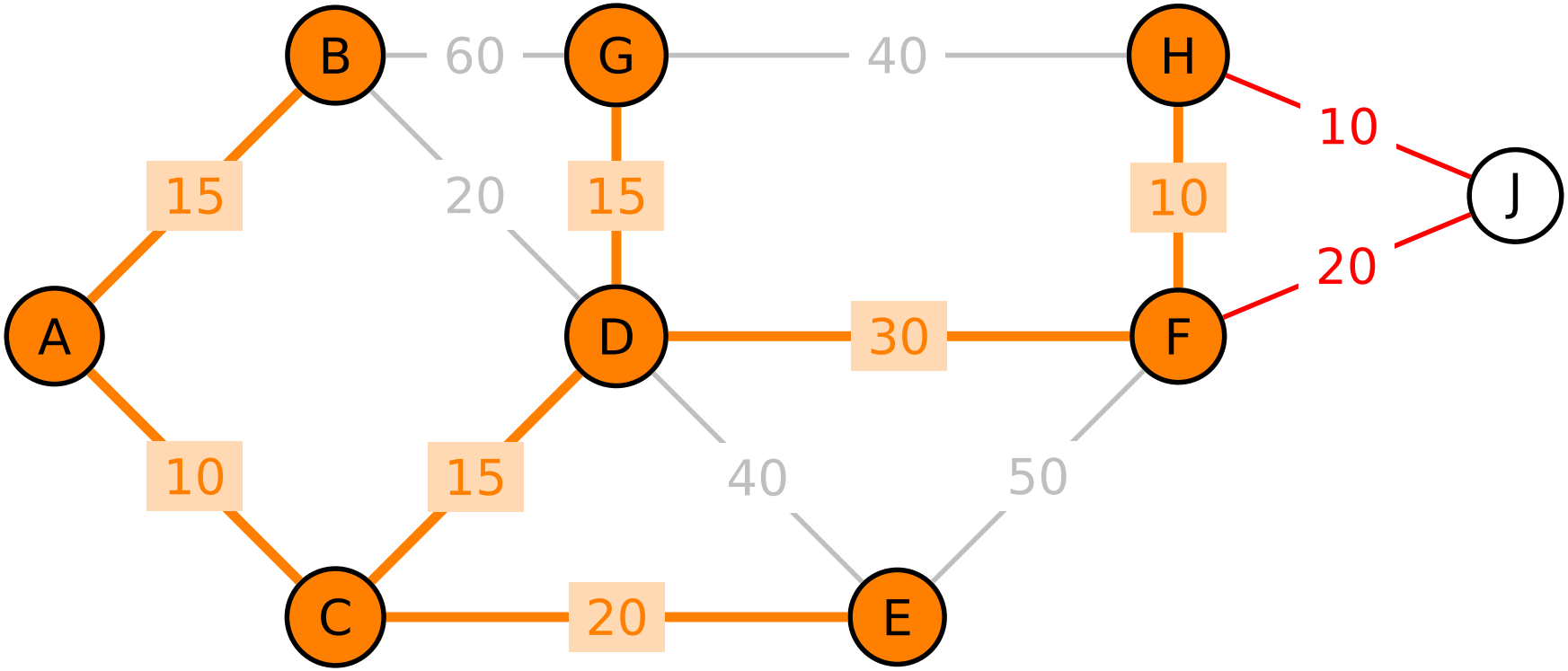
Выберем какую-нибудь из кратчайших дуг и добавим её к дереву. Снова находим варианты дуг, которые соединяют вершины дерева и неохваченные вершины



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

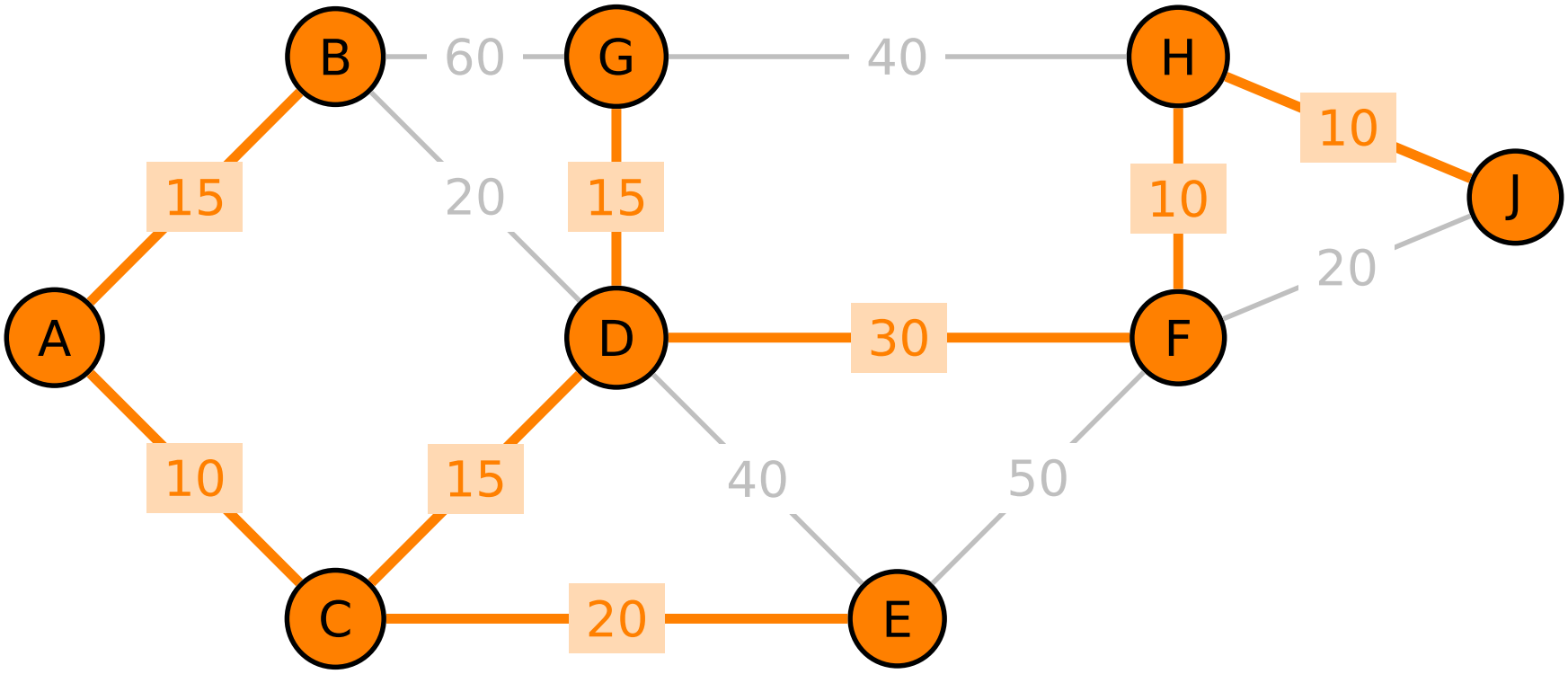
Выберем какую-нибудь из кратчайших дуг и добавим её к дереву. Снова находим варианты дуг, которые соединяют вершины дерева и неохваченные вершины



Построение минимального остовного дерева

Построим дерево для того же графа, что и раньше.

Все вершины попали в дерево, обработка закончена



Доказательство минимальности остовного дерева

Утверждение: жадный алгоритм (выбор на каждом шаге дуги с минимальной длиной) позволяет построить минимальное остовное дерево.

Доказательство (Jarnik, 1930; Prim, 1957; Dijkstra, 1959):

1. Пусть имеется некоторое минимальное остовное дерево T_m, которое не совпадает с получаемым алгоритмом Прима деревом T (если бы они всегда совпадали, утверждение теоремы было бы доказано). Пусть при построении дерева T на каком-то шаге выбрана дуга E, не входящая в T_m. Назовём концы этой дуги a и b; вершины, входящие в построенное на этом этапе дерево — V, а не входящие — $\overline{V}$.
2. В минимальном дереве T_m a и b соединяются некоторым путём из вершин, так как T_m — связный граф, причём по п. 1 E не входит в этот путь.

Доказательство минимальности остовного дерева (2)

3. Путь, соединяющий a и b, обязан где-то пройти из V в $\overline{V}$, причём не через дугу E. Назовём ту дугу этого пути, которая проходит из V в $\overline{V}$, F.
4. Раз жадный алгоритм выбрал не дугу F, а дугу E, то длина E меньше или равна длине F. Проверим, может ли длина E быть меньше, чем у F.
5. Если E короче F, выкинем из оптимального дерева T_m дугу F и добавим E. Полученное дерево по-прежнему соединяет все вершины графа, не имеет циклов, а сумма его длин меньше, чем у T_m. Это значит, что T_m не является оптимальным, что противоречит условию. Значит, длины дуг E и F равны.
6. Это значит, что суммы длин дуг T_m и T равны, т.е. любые различия в них не влияют на сумму, т.е. T тоже минимальное.

Кластеризация через минимальное остовное дерево

Вопрос: как его использовать для кластеризации?

Кластеризация через минимальное остовное дерево

Вопрос: как его использовать для кластеризации?
Ответ: вариант 1: убрать все дуги с длиной меньше некоего наперёд заданного порога T, связные компоненты — кластеры;
вариант 2: использовать его как готовую иерархию кластеров: если длина максимальной дуги T_{max}, получим кластеры первого уровня, убрав все такие дуги; в каждом кластере повторим то же самое, получим иерархию.

Достоинство: простота.
Недостаток: работает долго (надо считать много попарных расстояний).

Ещё одно пояснение по precision и recall

Пусть надо просеять песок, чтобы отделить его от примесей.
Классификатор — система, просеивающая песок.
Вопрос: что означают TP,TN,FP,FN,precision,recall?

Ещё одно пояснение по precision и recall

Пусть надо просеять песок, чтобы отделить его от примесей.
Классификатор — система, просеивающая песок.
Вопрос: что означают TP,TN,FP,FN,precision,recall?
Ответ:
True Positive: песок, который был классифицирован как песок
True Negative: не песок (примесь), которая распознана как примесь
False Positive: не песок, ошибочно классифицированный как песок; **ошибка первого рода**
False Negative — песок, ошибочно классифицированный как не песок; **ошибка второго рода**

$$Precision = TP/(TP + FP); Recall = TP/(TP + FN)$$

Precision — **насколько чистый** будет песок после просеивания
Recall — **насколько весь** песок попал в просеяннный

Спектральная кластеризация

Идея: снизить размерность задачи кластеризации путём использования свойств графа, которым можно описать матрицу сходства.

Пусть сходство между точками — некоторая симметричная функция $s(z_i, z_j) = s(z_j, z_i)$, которая высока для близких точек и мала для дальних; обычно она максимальна для совпадающих точек ($s(z_i, z_i)$). Составим неориентированный граф, у которого вершины — все точки, а дуга между точками имеется, если сходство между ними больше некоторого наперёд заданного порога s_{min}.

Понятно, что в таком графе группы похожих точек будут иметь высокую связность, непохожие будут связаны слабо. Хорошо бы разрезать его на слабо связанные компоненты.

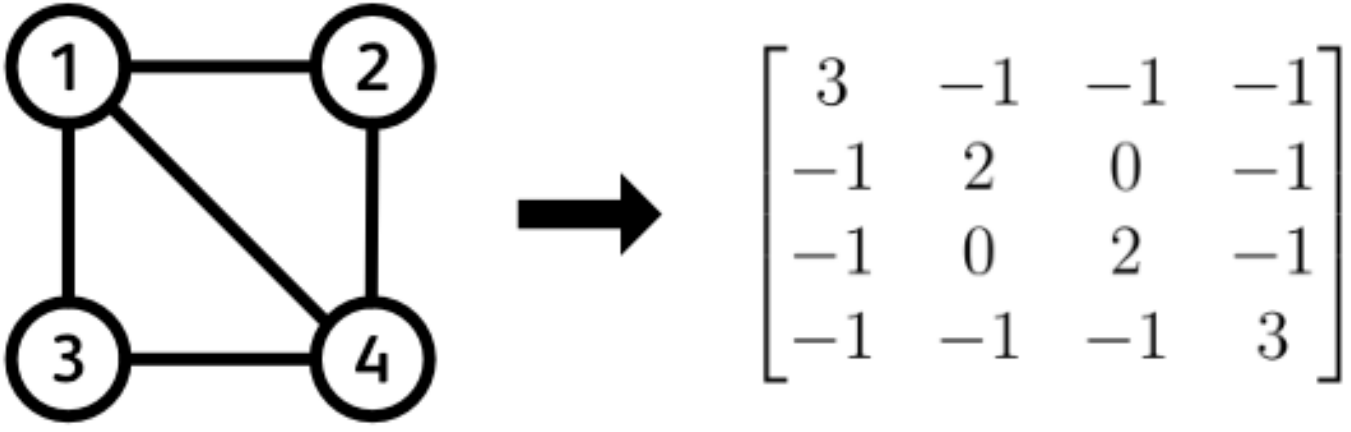
Спектральная кластеризация (2)

Обозначим полученный неориентированный граф $G = (V, E)$, где V (vertices) — вершины, E (edges) — дуги. Всего вершин столько же, сколько точек — N.

Степень вершины V — число дуг, сходящихся в этой вершине. **Матрица степени** графа D — диагональная матрица $N \times N$, составленная из степеней вершин.

Матрица смежности графа $W(G)$ — матрица размера $N \times N$, в которой элемент $w_{ij} = 1$, если между вершинами z_i и z_j есть дуга, и 0, если её нет.

Лапласиан графа $L = D - W$:



$$\begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & 0 & -1 \\ -1 & 0 & 2 & -1 \\ -1 & -1 & -1 & 3 \end{bmatrix}$$

Спектральная кластеризация (2)

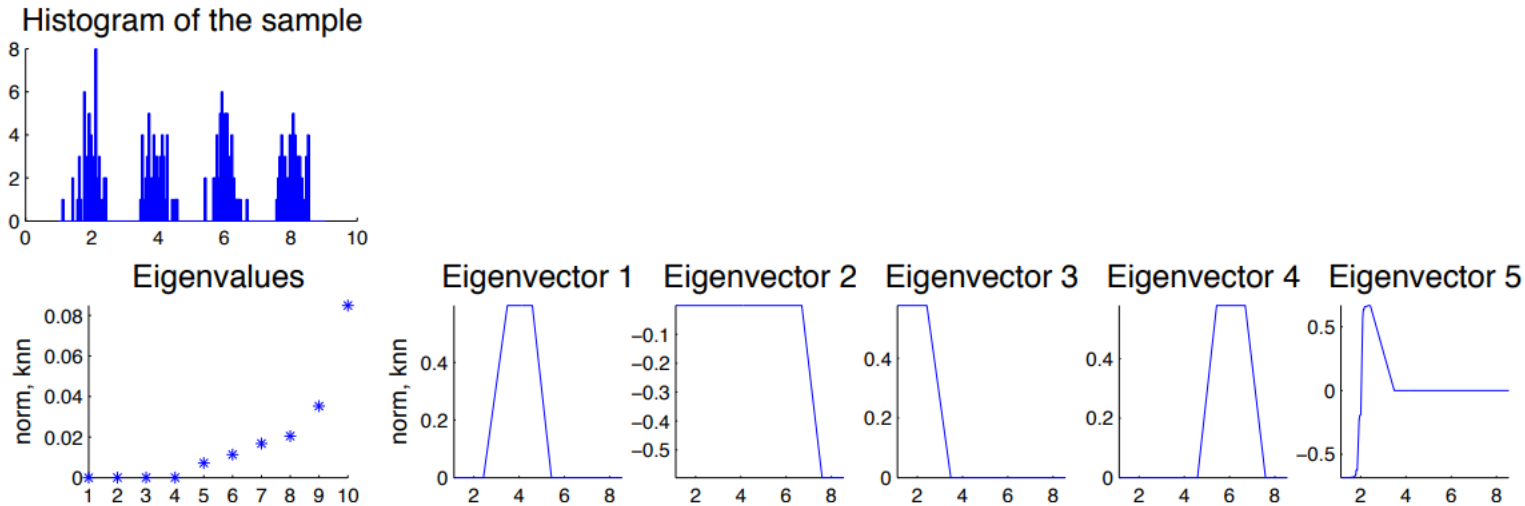
Можно доказать[4], что для лапласиана выполняются интересные свойства:

1. У него столько нулевых собственных значений, сколько связных компонентов в графе, причём каждый собственный вектор с нулевым собственным значением выделяет одну связную компоненту: координаты вектора по ней ближе друг к другу, чем к другим. Т.е. можно сразу выделить часть кластеров — связные компоненты, если нулевых собственных значений больше одного. Пусть это уже сделано, и нулевое собственное значение K одно.
2. Внутри связной компоненты есть хотя бы одно нулевое собственное значение, его собственный вектор единичный (все его координаты совпадают).
3. Если выбрать одно или несколько наименьших собственных значений и кластеризовать точки, пользуясь координатами в соответствующих векторах, а не оригинальными признаками, можно разделить граф на плотно связные компоненты.

[4] Ulrike von Luxburg. A Tutorial on Spectral Clustering. https://arxiv.org/abs/0711.0189

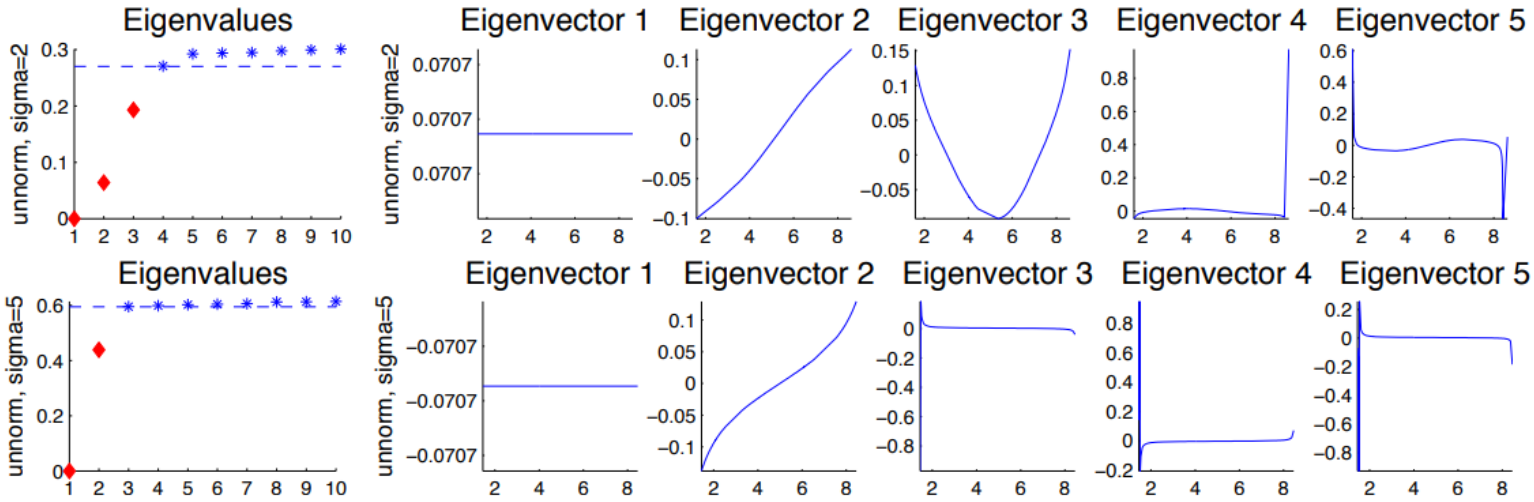
Спектральная кластеризация: связные компоненты

Если взять одномерные данные (смесь 4 нормальных распределений, гистограмма показана сверху слева) и представить их графом из нескольких отдельных компонентов, то собственные векторы нулевых собственных значений соответствуют несвязным компонентам:



Спектральная кластеризация: разделение компонентов

Если те же данные представить графом с большей связностью, то несколько минимальных собственных векторов позволят разделить компоненты:



Спектральная кластеризация: достоинства и недостатки

Математическая сущность: проекция данных в пространство более низкой размерности, в котором отделимость выше.

Достоинства: можно варьировать подходы благодаря использованию разных ядер, разделять линейно неотделимые данные, устойчивость к многомерным данным.

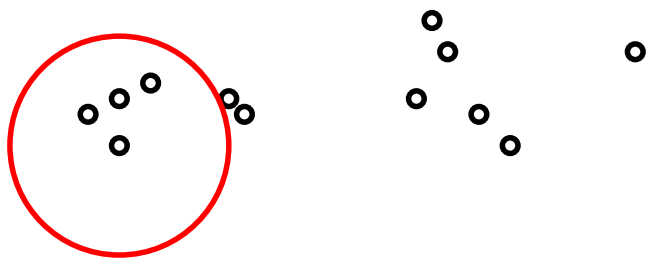
Недостатки: много гиперпараметров (ядро и др.), высокая вычислительная сложность.

Кластеризация через формальные элементы

FOREL (Загоруйко,Ёлкина, 1967)

Пусть известны попарные расстояния между всеми элементами выборки $\rho(z_i, z_j)$. Методы, которые используют главным образом эти расстояния, называются **метрическими**.

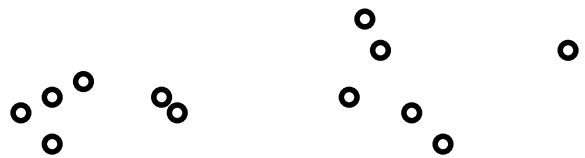
Идея: выделять группы близких элементов, т.е. тех, которые находятся ближе к какому-то одному из них, чем некоторый фиксированный радиус R.



Алгоритм поиска очередного кластера

1. зададимся некоторым наперёд заданным радиусом R;
2. выберем какой-либо элемент, не включенный в кластеры, и построим сферу с центром в нём и радиусом R. В неё попадут некоторые элементы выборки;
3. рассчитаем «центр» этих элементов;
4. построим сферу того же радиуса вокруг этого «центра»;
5. пока элементы, входящие в эту сферу, изменяются, переходим к пункту 3, иначе считаем эти элементы входящими в один кластер и возвращаем его.

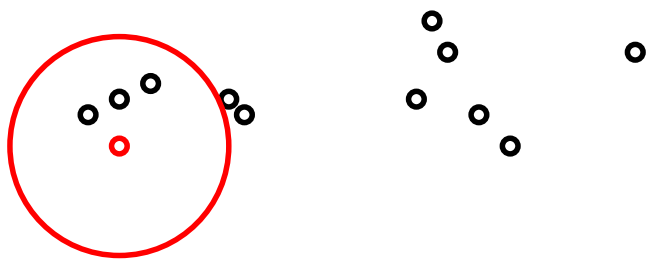
Доказано, что этот процесс сходится за конечное число шагов.



Алгоритм поиска очередного кластера

1. зададимся некоторым наперёд заданным радиусом R;
2. выберем какой-либо элемент, не включенный в кластеры, и построим сферу с центром в нём и радиусом R. В неё попадут некоторые элементы выборки;
3. рассчитаем «центр» этих элементов;
4. построим сферу того же радиуса вокруг этого «центра»;
5. пока элементы, входящие в эту сферу, изменяются, переходим к пункту 3, иначе считаем эти элементы входящими в один кластер и возвращаем его.

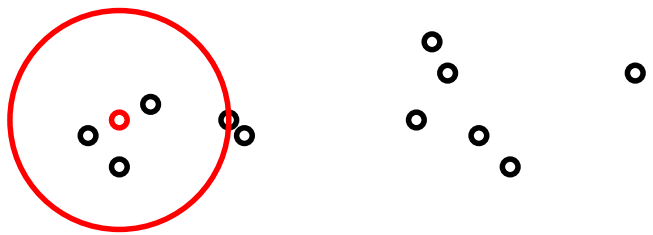
Доказано, что этот процесс сходится за конечное число шагов.



Алгоритм поиска очередного кластера

1. зададимся некоторым наперёд заданным радиусом R;
2. выберем какой-либо элемент, не включенный в кластеры, и построим сферу с центром в нём и радиусом R. В неё попадут некоторые элементы выборки;
3. рассчитаем «центр» этих элементов;
4. построим сферу того же радиуса вокруг этого «центра»;
5. пока элементы, входящие в эту сферу, изменяются, переходим к пункту 3, иначе считаем эти элементы входящими в один кластер и возвращаем его.

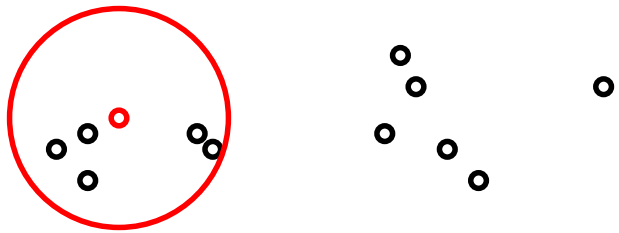
Доказано, что этот процесс сходится за конечное число шагов.



Алгоритм поиска очередного кластера

1. зададимся некоторым наперёд заданным радиусом R;
2. выберем какой-либо элемент, не включенный в кластеры, и построим сферу с центром в нём и радиусом R. В неё попадут некоторые элементы выборки;
3. рассчитаем «центр» этих элементов;
4. построим сферу того же радиуса вокруг этого «центра»;
5. пока элементы, входящие в эту сферу, изменяются, переходим к пункту 3, иначе считаем эти элементы входящими в один кластер и возвращаем его.

Доказано, что этот процесс сходится за конечное число шагов.



Формальный элемент

«Центр» кластера называется формальным элементом.

Вопрос: почему «центр» взят в кавычки? Почему нельзя просто сказать «точка с усреднёнными значениями координат»?

Формальный элемент

«Центр» кластера называется формальным элементом.

Вопрос: почему «центр» взят в кавычки? Почему нельзя просто сказать «точка с усреднёнными значениями координат»?
Ответ: потому что не всегда эти координаты известны (см. случай с ДНК, белками, текстами и т.п.). Тогда вместо усреднённого значения координат придётся брать тот элемент z_i, который ближе всего ко всем другим, например с минимальной суммой расстояний до всех других N элементов

$$\sum_{j=1}^{N} \rho(z_i, z_j)$$

Вопрос: почему элемент «формальный»?

Формальный элемент

«Центр» кластера называется формальным элементом.

Вопрос: почему «центр» взят в кавычки? Почему нельзя просто сказать «точка с усреднёнными значениями координат»?
Ответ: потому что не всегда эти координаты известны (см. случай с ДНК, белками, текстами и т.п.). Тогда вместо усреднённого значения координат придётся брать тот элемент z_i, который ближе всего ко всем другим, например с минимальной суммой расстояний до всех других N элементов

$$\sum_{j=1}^{N} \rho(z_i, z_j)$$

Вопрос: почему элемент «формальный»?
Ответ: потому что он может отсутствовать в выборке, если рассчитан усреднением координат.

Дальнейшие шаги метода формального элемента

Когда вся выборка разбита на такие шарообразные кластеры, можно или считать задачу решённой, или дополнительно кластеризовать уже их; так как их количество мало по сравнению с размером выборки, это можно сделать быстро.

Достоинства: можно настраивать результат, меняя R, в том числе в процессе работы; форма кластеров может быть произвольной; можно использовать как предобработку для других методов.

Недостатки: большое время выполнения, особенно если неизвестны координаты прецедентов (т.е. на каждом шаге придётся считать все попарные расстояния в текущем кластере).

MeanShift

Идея похожа на метод формального элемента, но применяет произвольное ядро вместо шара:

1. зададимся функцией ядра $K(z_i, z_j) = K(z_i - z_j)$ — часто используется гауссиана;
2. выбираем случайную начальную центральную точку кластера z_c;
3. рассчитываем центроид окна, взвешенного с помощью выбранного ядра: $m(z_c) = \frac{\sum\limits_{z \in Z} K(z - z_c) z}{\sum\limits_{z \in Z} K(z - z_c)}$;
4. сдвигаем z_c в $m(z_c)$ (или в сторону $m(z_c)$);
5. если сдвиг оказался меньше порогового значения, заканчиваем, иначе переходим к шагу 3.

Достоинства и недостатки — см. метод формального элемента.

Affinity propagation

Идея: провести множество итераций решений, когда каждая точка решает, какая точка из всей выборки больше всего годится на роль основы её кластера. Анализируются два фактора:

- насколько данная точка хочет видеть основой кластера другую;
- насколько другая точка хочет стать основой кластера данной.

При этом изначально точки не стремятся быть основами кластеров (чтобы они не голосовали за себя, т.е. чтобы кластеры не вырождались в единичные точки).
Когда голосования сходятся к постоянному результату, задача решена.

Формально для описания процесса вводят два попарно рассчитываемых значения — **ответственность** (responsibility, r) и **доступность** (availability, a).

Affinity propagation: ответственность

Функции двух точек $r(z_i, z_j)$ (ответственность) и $a(z_i, z_j)$ (доступность) рассчитываются итеративно. До начала итераций все r и a приравнивают к нулю. Вместо функции расстояния $\rho(z_i, z_j)$ обычно вводят функцию схожести:

$$s(z_i, z_j) = \begin{cases} -\rho(z_i, z_j), i \neq j \\ -const, i = j \end{cases}$$

Ответственность:

$$r(z_i, z_j) = s(z_i, z_j) - \max(a(z_i, z_k) + s(z_i, z_k)), z_k \in Z, k \neq j$$

Физический смысл:

1. чем больше схожесть точек i и j, тем больше i хочет возложить ответственность на j;
2. чем больше сходство j и самой похожей на него точки, кроме i, тем меньше i хочет возложить ответственность на j.

Affinity propagation: доступность

После ответственности рассчитывается доступность — насколько каждая точка i готова быть основой кластера точки j:

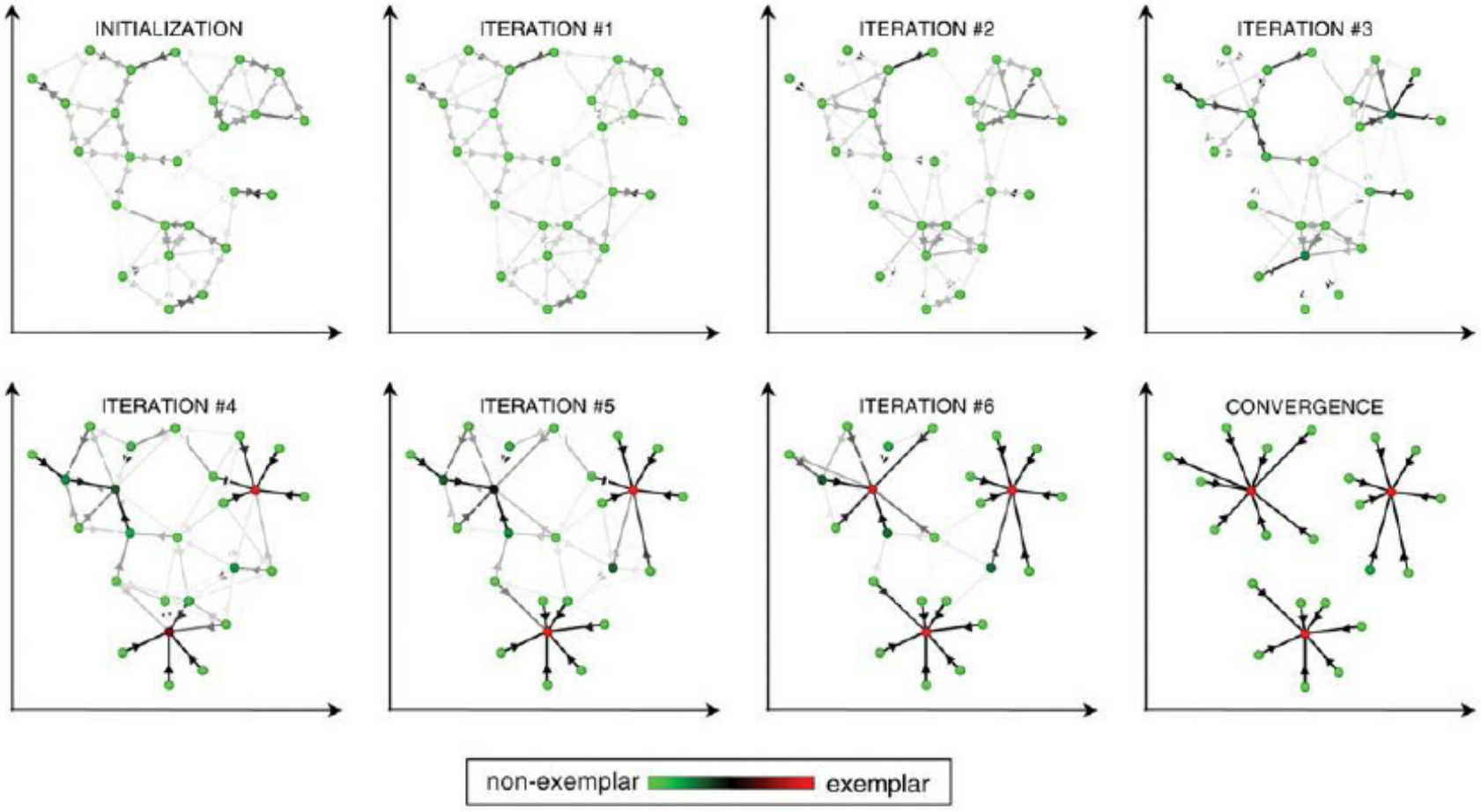
$$a(z_i, z_j) = \min(0, r(z_j, z_j) + \sum_{k \neq j, k \neq i} \max(0, r(z_k, z_j))), i \neq j$$

$$a(z_i, z_i) = \sum_{j \neq i} \max(0, r(z_j, z_i))$$

Физический смысл:

1. доступность для самой себя — сумма полученных положительных r: «голосов, отданных за точку»;
2. доступность для других: если другая точка плохо способна представлять сама себя (низкая $r(z_j, z_j)$) и другие не хотят, чтобы она представляла их, её доступность будет выше.

Affinity propagation: пример



Affinity propagation: свойства

Можно сказать, что разные объекты одного кластера — на самом деле экземпляры одного объекта.

Изначально все значения r и a нулевые, потом они обновляются итеративно. Для борьбы с неустойчивостями часто новые значения a и r не заменяют старые, а двигают старые в свою сторону.

Достоинство: высокое качество; недостаток: низкая скорость.

DBSCAN

Density-based spatial clustering of applications with noise
Идея:

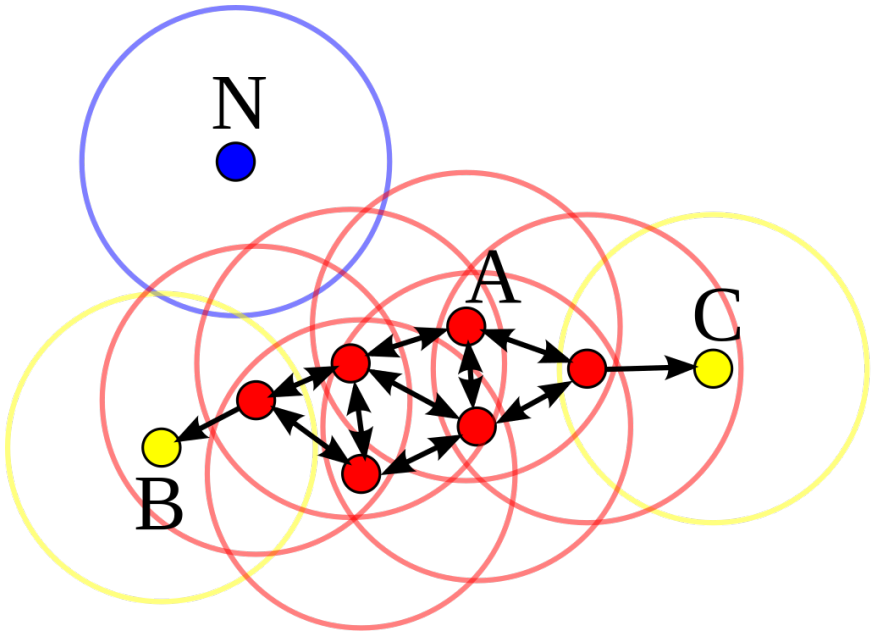
1. если поблизости от объекта имеется много соседей, то его окрестность можно назвать плотно заполненной;
2. если какие-то объекты расположены плотно, то они, скорее всего, находятся в одном кластере;
3. если область произвольной формы плотно заполнена объектами, она — единый кластер.

Более формально: пусть снова известны попарные расстояния от каждого объекта до каждого $\rho(z_i, z_j)$ и, кроме того, заданы радиус окрестности ε и пороговое число соседей N. Разделим точки на три вида:

1. коренные: в их ε-окрестности N или больше соседей;
2. краевые: в их ε-окрестности есть коренные точки;
3. изолированные: прочие.

DBSCAN: типы точек

Пусть N=4; тогда красные точки — коренные, жёлтые — краевые, синие — изолированные.



Соединим все близкие друг другу коренные точки в кластеры: если от одной коренной точки до другой есть цепь близких друг к другу коренных точек, то они — в одном кластере.

Краевые точки добавим к кластеру какой-то из соседних коренных; изолированные либо не трогаем, либо обрабатываем отдельно.

DBSCAN: достоинства и недостатки

Достоинства:

- форма кластера не важна;
- можно оставить часть точек вне кластеров, т.е. определить выбросы;
- работает в одну стадию, без итераций.

Недостатки:

- часто соединяет изолированные кластеры, если случайно найдётся узкая перемычка между ними;
- параметры надо подбирать скользящим контролем, при большой размерности наглядность теряется.

OPTICS

Ordering Points To Identify the Clustering Structure [5]

Идея: модифицировать DBSCAN, чтобы учесть колебания плотности точек в разных областях пространства.

Напоминание об идее DBSCAN: если задаться пороговым расстоянием между точками внутри кластера ε и числом соседей N, можно выделить точки, имеющие в окрестности радиуса ε N или больше соседей, и считать их коренными точками кластеров (их основой).

[5] Ankerst M., Breunig M., Kriegel H., Sander J. OPTICS: Ordering Points To Identify the Clustering Structure // ACM SIGMOD international conference on Management of data. ACM Press. pp. 49–60. 1999.

Параметры и выполнение OPTICS

Зададимся минимальным размером кластера M и пороговым расстоянием ε, после этого:

1. для каждой точки найдём корневое расстояние CD (core distance): минимальный размер её окрестности, в которой лежат $M-1$ соседей (т.е. M точек, включая текущую). Если оно получится больше ε, больше её не рассматриваем — она слишком далека от остальных;
2. для каждой пары оставшихся точек найдём расстояние достижимости RD (reachability distance): $RD(p_1, p_2) = max(CD(p_2), distance(p_1, p_2))$. Смысл: если считать, что p_2 будет коренной точкой, то $RD(p_1, p_2)$ — самое маленькое расстояния, чтобы p_1 был бы в том же кластере
3. если $RD < \varepsilon$, считать, что точки пары находятся в одном кластере.

Сравнение DBSCAN и OPTICS

В целом результаты после хорошего подбора коэффициентов часто похожи.

Один запуск DBSCAN срабатывает быстрее одного запуска OPTICS.

Для DBSCAN сложнее подбирать параметры (их два), поэтому суммарное время подготовки с учётом повторных запусков обычно лучше у OPTICS.